

Graphs - DFS & Connected Components

Fundamental Algorithms

Rodica Potolea, Camelia Lemnaru and Ciprian Oprea

Technical University of Cluj-Napoca
Computer Science Department

Agenda

- 1 Depth First Search (DFS)
- 2 Connected components
 - Connected components
 - Strongly connected components



Agenda

- 1 Depth First Search (DFS)
- 2 Connected components
 - Connected components
 - Strongly connected components



Depth First Search (1)

- similar to BFS, the queue is replaced by a stack
 - the stack is implicit with recursive calls
- instead of enqueueing the neighbors, we make a recursive call for each of them
- keep the color representation
 - WHITE - unvisited (all vertices are WHITE at first)
 - GRAY - under visitation (all GRAY vertices are on the stack)
 - BLACK - visited (in the end end, all nodes become BLACK)
- the color clusters define a vertical boundary between visited/unvisited vertices



Depth First Search (2)

- keep the parent attribute (π) - a reference to the vertex from where we reached the current vertex
- add the attributes
 - d - **d**iscovery time
 - f - **f**inish time
- π , d and f provide useful information for other algorithms
 - edge classification
 - topological sort



The DFS Algorithm

DFS(G)

```

1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
  
```

DFS-VISIT(G, u)

```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



The DFS Algorithm

DFS(G)

```

1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
  
```

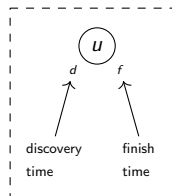
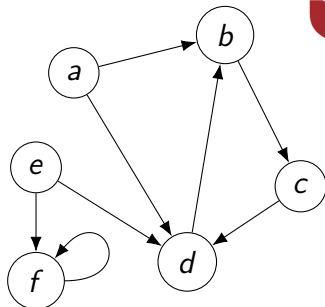
DFS-VISIT(G, u)

```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```

DFS

forest:





The DFS Algorithm

DFS(G)

```

1  for each vertex  $u \in G.V$ 
2       $u.color = WHITE$ 
3       $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == WHITE$ 
7          DFS-VISIT( $G, u$ )
  
```

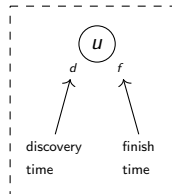
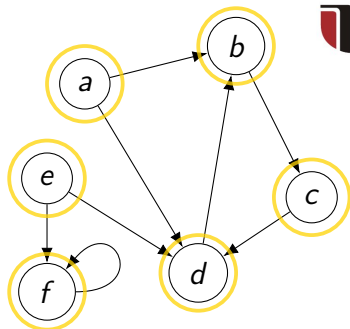
DFS-VISIT(G, u)

```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == WHITE$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```

DFS

forest:





The DFS Algorithm

DFS(G)

```

1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4  time = 0
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
  
```

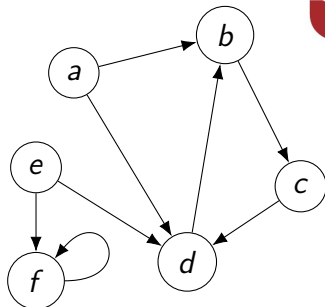
DFS-VISIT(G, u)

```

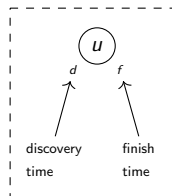
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```

DFS

forest:



$time = 0$





The DFS Algorithm

DFS(G)

```

1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6    if  $u.color == WHITE$ 
7      DFS-VISIT( $G, u$ )
  
```

DFS-VISIT(G, u)

```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```

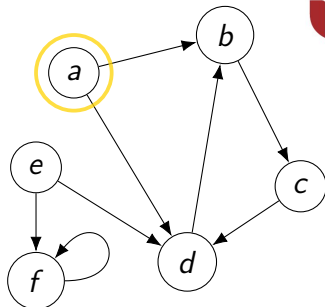


stack

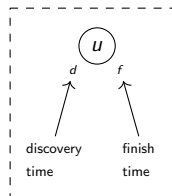


DFS

forest:



$time = 0$





The DFS Algorithm

DFS(G)

```

1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
  
```

DFS-VISIT(G, u)

```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```

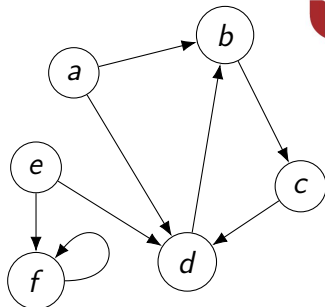


stack

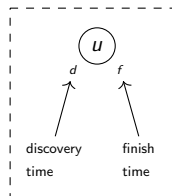


DFS

forest:



$time = 1$





The DFS Algorithm

DFS(G)

```

1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
  
```

DFS-VISIT(G, u)

```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```

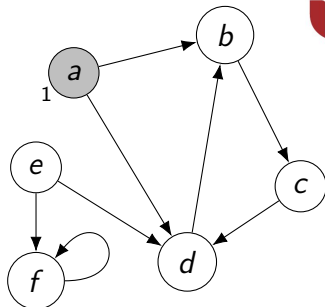


stack

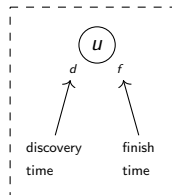


DFS

forest:



$time = 1$





The DFS Algorithm

DFS(G)

```

1  for each vertex  $u \in G.V$ 
2       $u.color = WHITE$ 
3       $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == WHITE$ 
7          DFS-VISIT( $G, u$ )
  
```

DFS-VISIT(G, u)

```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == WHITE$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```

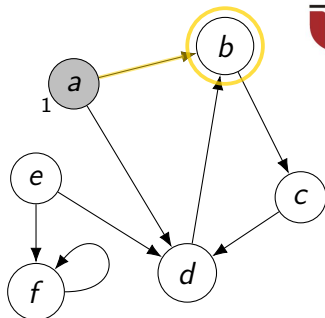


stack

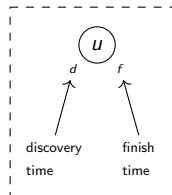


DFS

forest:



$time = 1$





The DFS Algorithm

DFS(G)

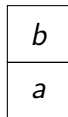
```

1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
  
```

DFS-VISIT(G, u)

```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```

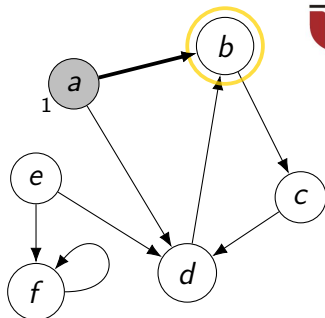


stack

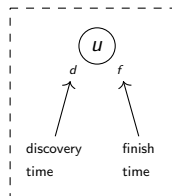


DFS

forest:



$time = 1$





The DFS Algorithm

DFS(G)

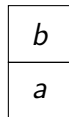
```

1  for each vertex  $u \in G.V$ 
2       $u.color = WHITE$ 
3       $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == WHITE$ 
7          DFS-VISIT( $G, u$ )
  
```

DFS-VISIT(G, u)

```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == WHITE$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```

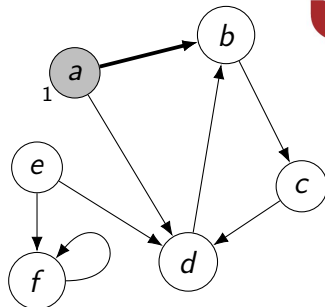


stack

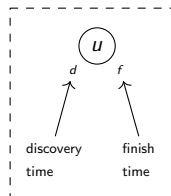


DFS

forest:



$time = 2$





The DFS Algorithm

DFS(G)

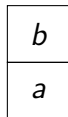
```

1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6    if  $u.color == WHITE$ 
7      DFS-VISIT( $G, u$ )
  
```

DFS-VISIT(G, u)

```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```

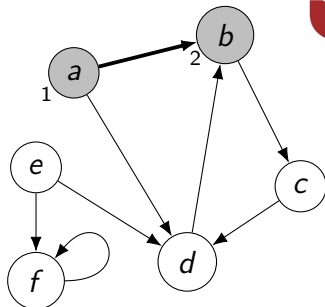


stack

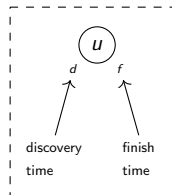


DFS

forest:



$time = 2$





The DFS Algorithm

DFS(G)

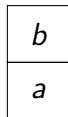
```

1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6    if  $u.color == WHITE$ 
7      DFS-VISIT( $G, u$ )
  
```

DFS-VISIT(G, u)

```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```

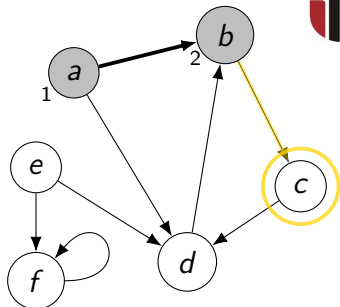


stack

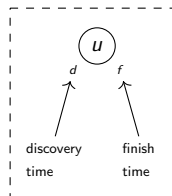


DFS

forest:



$time = 2$





The DFS Algorithm

DFS(G)

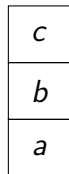
```

1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
  
```

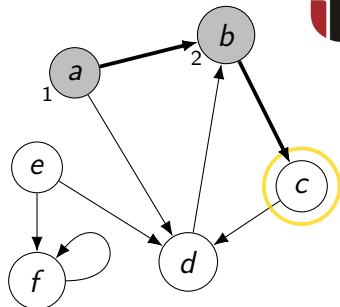
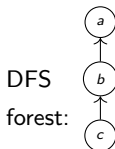
DFS-VISIT(G, u)

```

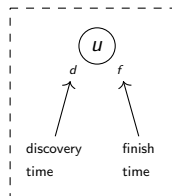
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



stack



$time = 2$





The DFS Algorithm

DFS(G)

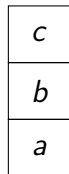
```

1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6    if  $u.color == WHITE$ 
7      DFS-VISIT( $G, u$ )
  
```

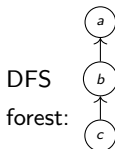
DFS-VISIT(G, u)

```

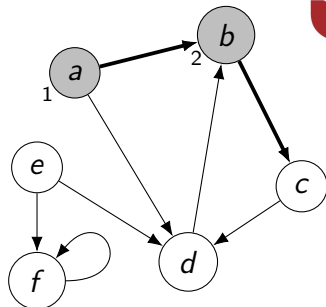
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



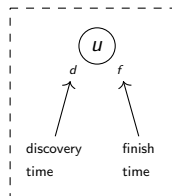
stack



DFS
forest:



$time = 3$





The DFS Algorithm

DFS(G)

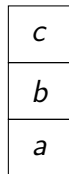
```

1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6    if  $u.color == WHITE$ 
7      DFS-VISIT( $G, u$ )
  
```

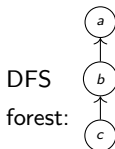
DFS-VISIT(G, u)

```

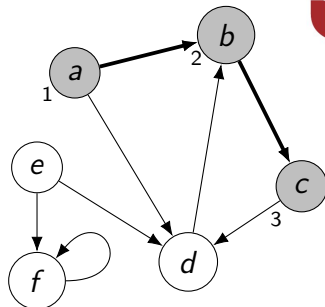
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



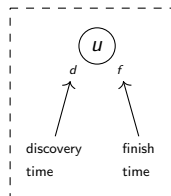
stack



DFS
forest:



$time = 3$





The DFS Algorithm

DFS(G)

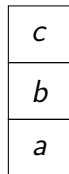
```

1  for each vertex  $u \in G.V$ 
2       $u.color = WHITE$ 
3       $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == WHITE$ 
7          DFS-VISIT( $G, u$ )
  
```

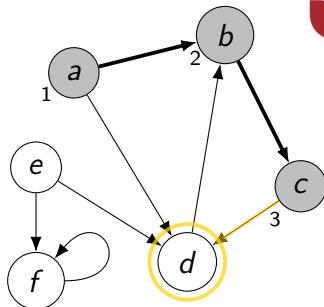
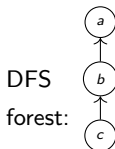
DFS-VISIT(G, u)

```

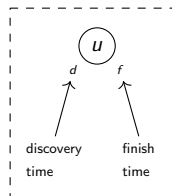
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == WHITE$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



stack



$time = 3$





The DFS Algorithm

DFS(G)

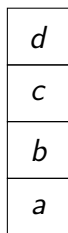
```

1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
  
```

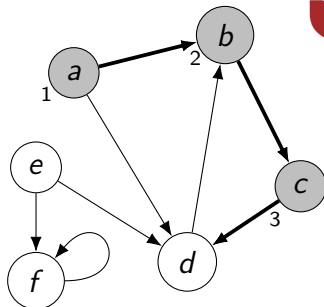
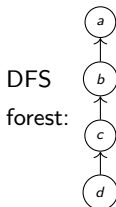
DFS-VISIT(G, u)

```

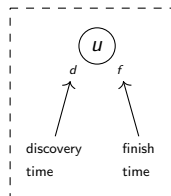
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



stack



$time = 3$





The DFS Algorithm

DFS(G)

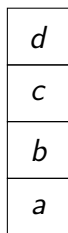
```

1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
  
```

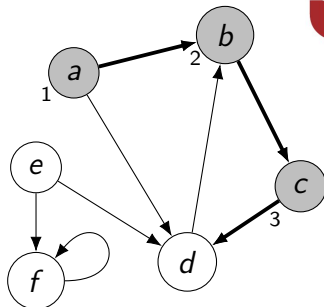
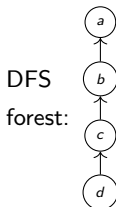
DFS-VISIT(G, u)

```

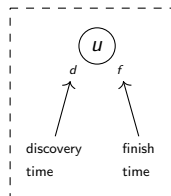
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



stack



$time = 4$





The DFS Algorithm

DFS(G)

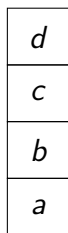
```

1  for each vertex  $u \in G.V$ 
2       $u.color = WHITE$ 
3       $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == WHITE$ 
7          DFS-VISIT( $G, u$ )
  
```

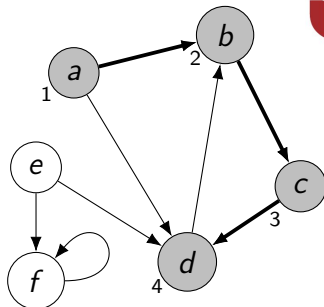
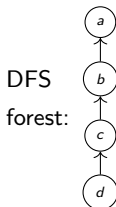
DFS-VISIT(G, u)

```

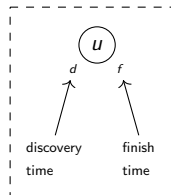
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == WHITE$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



stack



$time = 4$





The DFS Algorithm

DFS(G)

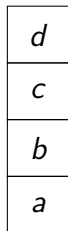
```

1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
  
```

DFS-VISIT(G, u)

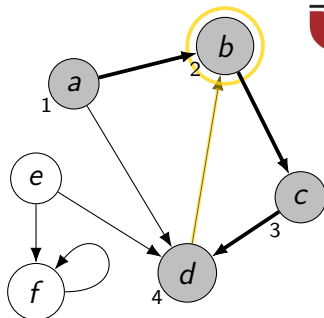
```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```

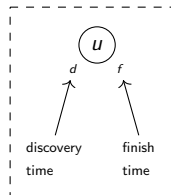


stack

DFS
forest:



$time = 4$





The DFS Algorithm

DFS(G)

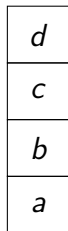
```

1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
  
```

DFS-VISIT(G, u)

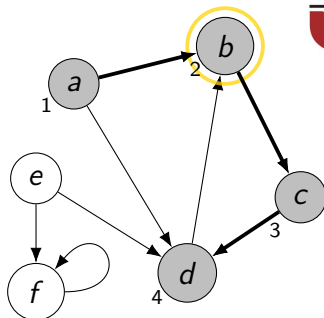
```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```

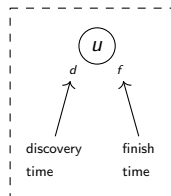


stack

DFS
forest:



$time = 4$





The DFS Algorithm

DFS(G)

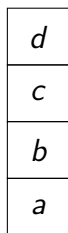
```

1  for each vertex  $u \in G.V$ 
2       $u.color = WHITE$ 
3       $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == WHITE$ 
7          DFS-VISIT( $G, u$ )
  
```

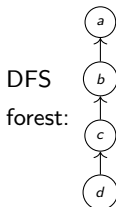
DFS-VISIT(G, u)

```

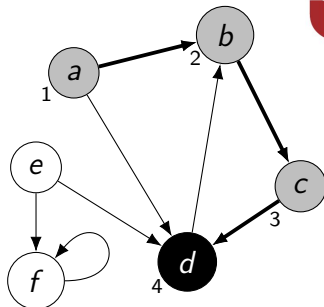
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == WHITE$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



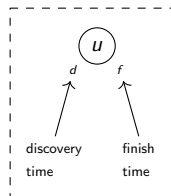
stack



DFS
forest:



$time = 4$





The DFS Algorithm

DFS(G)

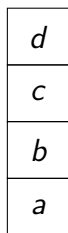
```

1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
  
```

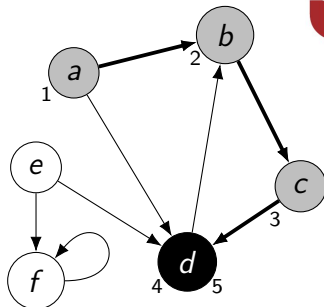
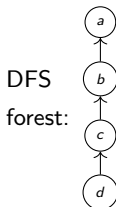
DFS-VISIT(G, u)

```

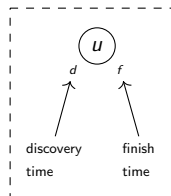
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



stack



$time = 5$





The DFS Algorithm

DFS(G)

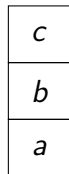
```

1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6    if  $u.color == WHITE$ 
7      DFS-VISIT( $G, u$ )
  
```

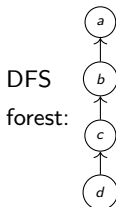
DFS-VISIT(G, u)

```

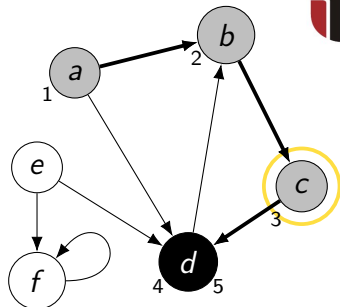
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



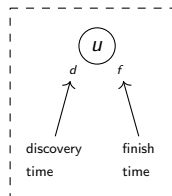
stack



DFS
forest:



$time = 5$





The DFS Algorithm

DFS(G)

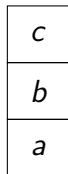
```

1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
  
```

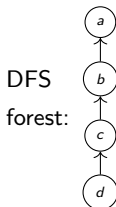
DFS-VISIT(G, u)

```

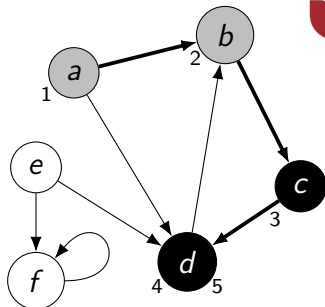
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



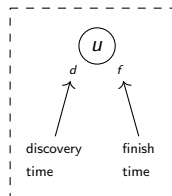
stack



DFS
forest:



$time = 5$





The DFS Algorithm

DFS(G)

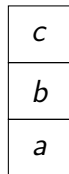
```

1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6    if  $u.color == WHITE$ 
7      DFS-VISIT( $G, u$ )
  
```

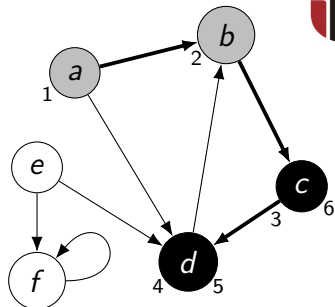
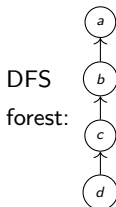
DFS-VISIT(G, u)

```

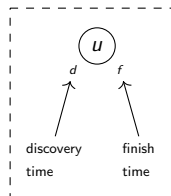
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



stack



$time = 6$





The DFS Algorithm

DFS(G)

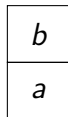
```

1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
  
```

DFS-VISIT(G, u)

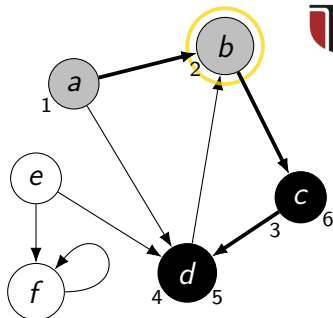
```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```

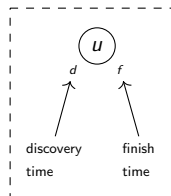


stack

DFS
forest:



$time = 6$





The DFS Algorithm

DFS(G)

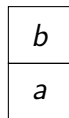
```

1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6    if  $u.color == WHITE$ 
7      DFS-VISIT( $G, u$ )
  
```

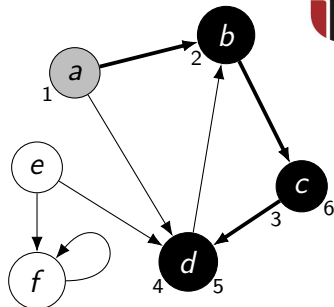
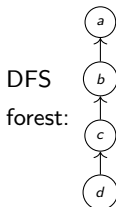
DFS-VISIT(G, u)

```

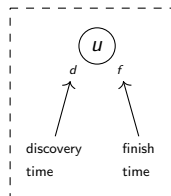
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



stack



$time = 6$





The DFS Algorithm

DFS(G)

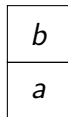
```

1  for each vertex  $u \in G.V$ 
2       $u.color = WHITE$ 
3       $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == WHITE$ 
7          DFS-VISIT( $G, u$ )
  
```

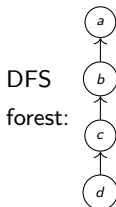
DFS-VISIT(G, u)

```

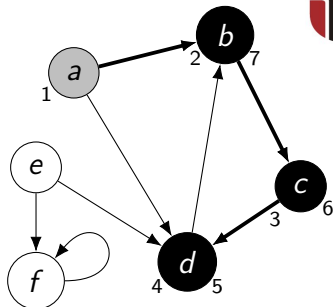
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == WHITE$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



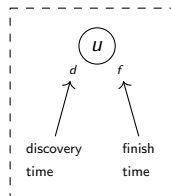
stack



DFS
forest:



$time = 7$





The DFS Algorithm

DFS(G)

```

1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
  
```

DFS-VISIT(G, u)

```

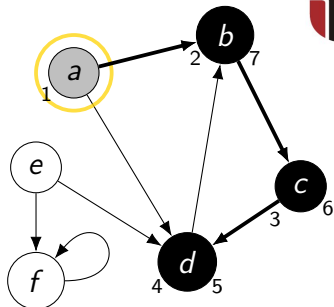
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



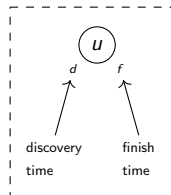
stack

DFS

forest:



$time = 7$





The DFS Algorithm

DFS(G)

```

1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6    if  $u.color == WHITE$ 
7      DFS-VISIT( $G, u$ )
  
```

DFS-VISIT(G, u)

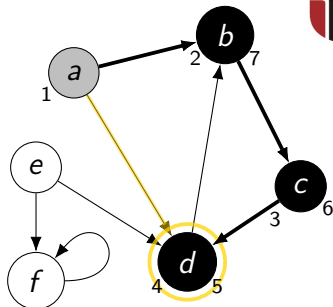
```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```

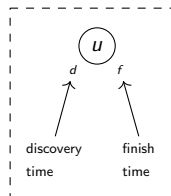


stack

DFS
forest:



$time = 7$





The DFS Algorithm

DFS(G)

```

1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
  
```

DFS-VISIT(G, u)

```

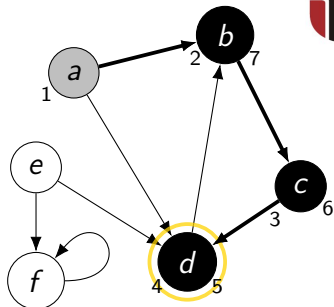
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



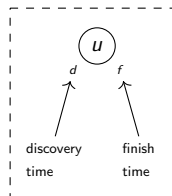
stack

DFS

forest:



$time = 7$





The DFS Algorithm

DFS(G)

```

1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6    if  $u.color == WHITE$ 
7      DFS-VISIT( $G, u$ )
  
```

DFS-VISIT(G, u)

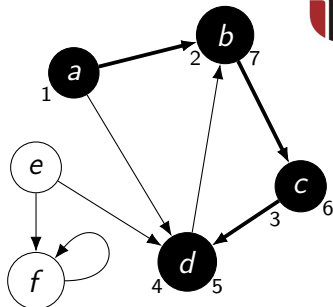
```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```

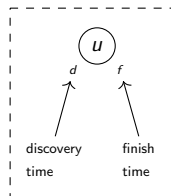


stack

DFS
forest:



$time = 7$





The DFS Algorithm

DFS(G)

```

1  for each vertex  $u \in G.V$ 
2       $u.color = WHITE$ 
3       $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == WHITE$ 
7          DFS-VISIT( $G, u$ )
  
```

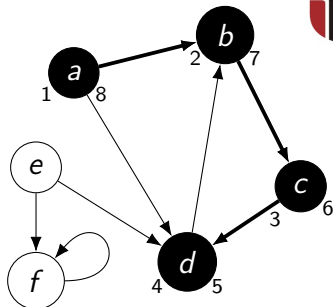
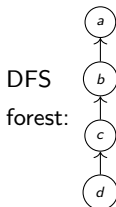
DFS-VISIT(G, u)

```

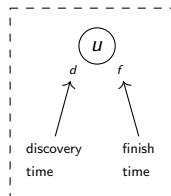
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == WHITE$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



stack



$time = 8$



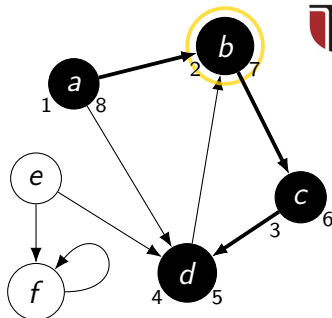


The DFS Algorithm

DFS(G)

```

1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
  
```



DFS-VISIT(G, u)

```

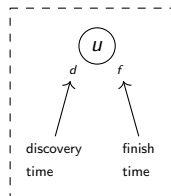
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```

stack

DFS
forest:



$time = 8$





The DFS Algorithm

DFS(G)

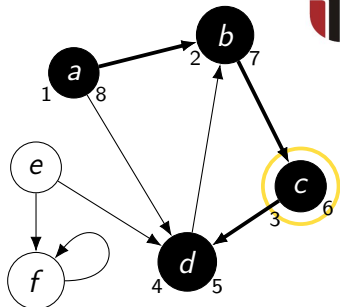
```

1  for each vertex  $u \in G.V$ 
2       $u.color = WHITE$ 
3       $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == WHITE$ 
7          DFS-VISIT( $G, u$ )
  
```

DFS-VISIT(G, u)

```

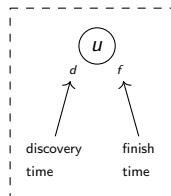
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == WHITE$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



stack

$time = 8$

DFS
forest:





The DFS Algorithm

DFS(G)

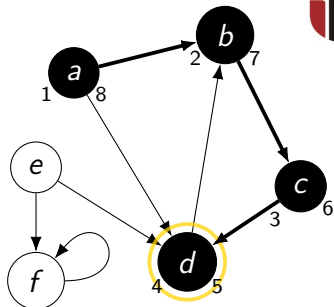
```

1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6    if  $u.color == WHITE$ 
7      DFS-VISIT( $G, u$ )
  
```

DFS-VISIT(G, u)

```

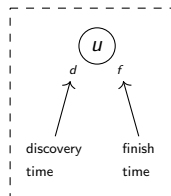
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



stack

$time = 8$

DFS
forest:





The DFS Algorithm

DFS(G)

```

1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6    if  $u.color == WHITE$ 
7      DFS-VISIT( $G, u$ )
  
```

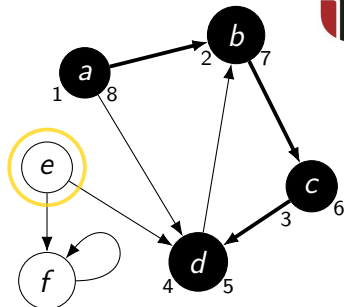
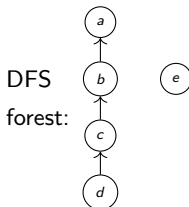
DFS-VISIT(G, u)

```

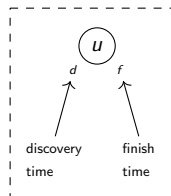
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



stack



$time = 8$





The DFS Algorithm

DFS(G)

```

1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6    if  $u.color == WHITE$ 
7      DFS-VISIT( $G, u$ )
  
```

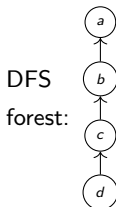
DFS-VISIT(G, u)

```

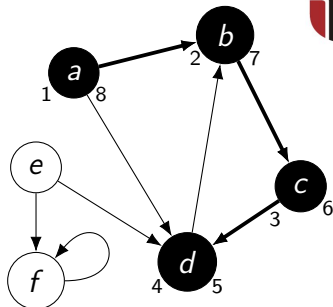
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



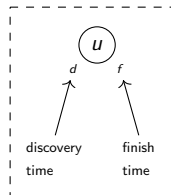
stack



DFS
forest:



$time = 9$





The DFS Algorithm

DFS(G)

```

1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6    if  $u.color == WHITE$ 
7      DFS-VISIT( $G, u$ )
  
```

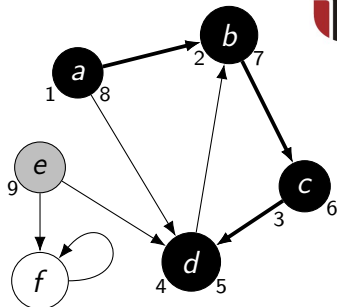
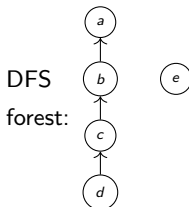
DFS-VISIT(G, u)

```

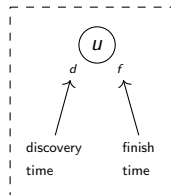
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



stack



$time = 9$





The DFS Algorithm

DFS(G)

```

1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6    if  $u.color == WHITE$ 
7      DFS-VISIT( $G, u$ )
  
```

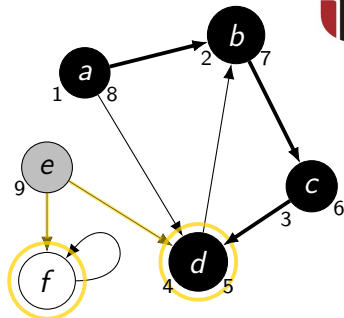
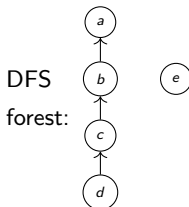
DFS-VISIT(G, u)

```

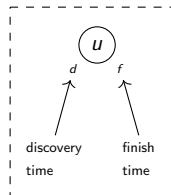
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



stack



$time = 9$





The DFS Algorithm

DFS(G)

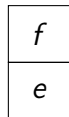
```

1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6    if  $u.color == WHITE$ 
7      DFS-VISIT( $G, u$ )
  
```

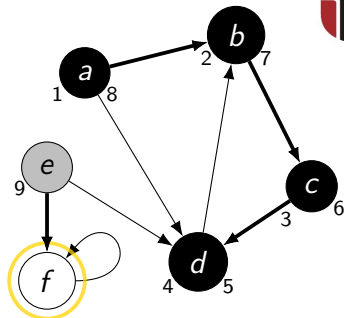
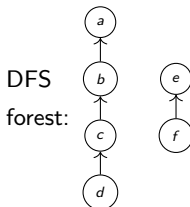
DFS-VISIT(G, u)

```

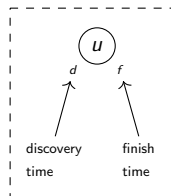
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



stack



$time = 9$





The DFS Algorithm

DFS(G)

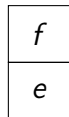
```

1  for each vertex  $u \in G.V$ 
2       $u.color = WHITE$ 
3       $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == WHITE$ 
7          DFS-VISIT( $G, u$ )
  
```

DFS-VISIT(G, u)

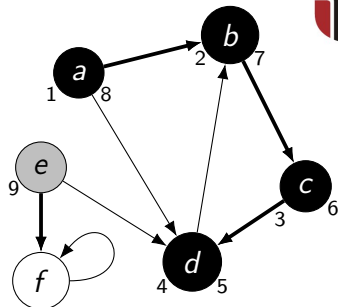
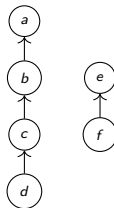
```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == WHITE$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```

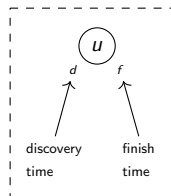


stack

DFS
forest:



$time = 10$





The DFS Algorithm

DFS(G)

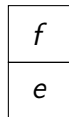
```

1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6    if  $u.color == WHITE$ 
7      DFS-VISIT( $G, u$ )
  
```

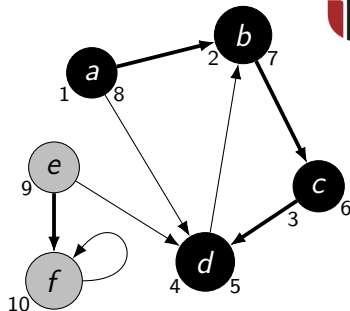
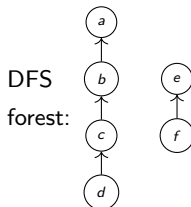
DFS-VISIT(G, u)

```

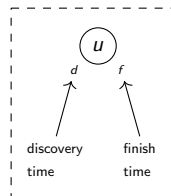
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



stack



$time = 10$





The DFS Algorithm

DFS(G)

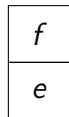
```

1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6    if  $u.color == WHITE$ 
7      DFS-VISIT( $G, u$ )
  
```

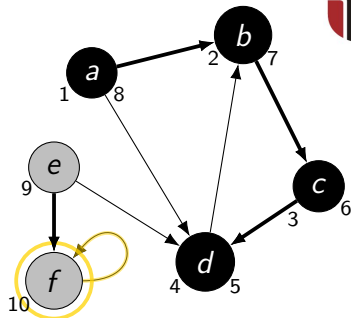
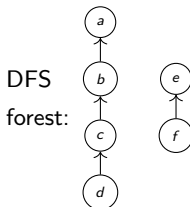
DFS-VISIT(G, u)

```

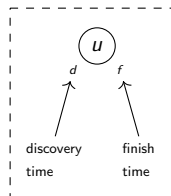
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



stack



$time = 10$





The DFS Algorithm

DFS(G)

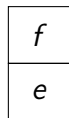
```

1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
  
```

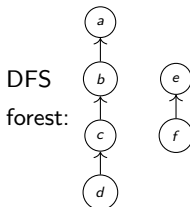
DFS-VISIT(G, u)

```

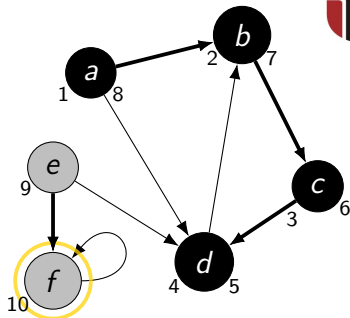
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



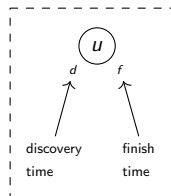
stack



DFS
forest:



$time = 10$





The DFS Algorithm

DFS(G)

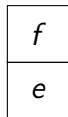
```

1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6    if  $u.color == WHITE$ 
7      DFS-VISIT( $G, u$ )
  
```

DFS-VISIT(G, u)

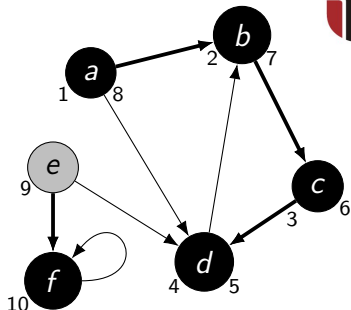
```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```

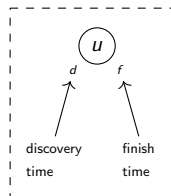


stack

DFS
forest:



$time = 10$





The DFS Algorithm

DFS(G)

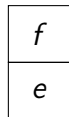
```

1  for each vertex  $u \in G.V$ 
2       $u.color = WHITE$ 
3       $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == WHITE$ 
7          DFS-VISIT( $G, u$ )
  
```

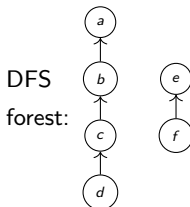
DFS-VISIT(G, u)

```

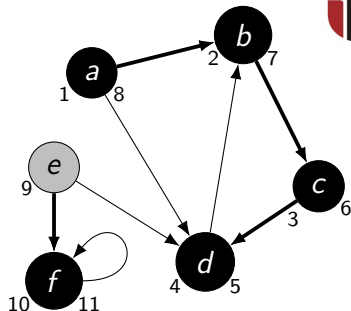
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == WHITE$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



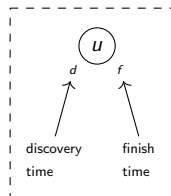
stack



DFS
forest:



$time = 11$





The DFS Algorithm

DFS(G)

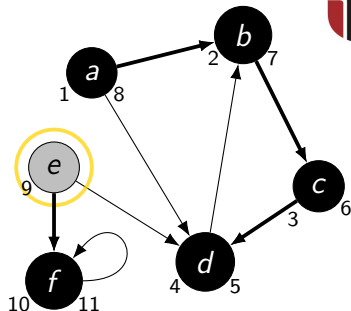
```

1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6    if  $u.color == WHITE$ 
7      DFS-VISIT( $G, u$ )
  
```

DFS-VISIT(G, u)

```

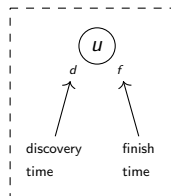
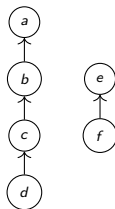
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



stack

$time = 11$

DFS
forest:





The DFS Algorithm

DFS(G)

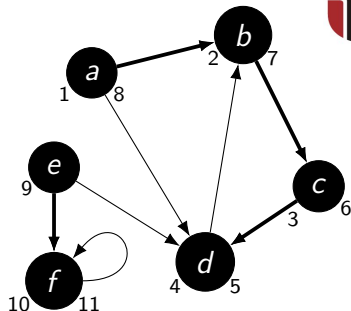
```

1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6    if  $u.color == WHITE$ 
7      DFS-VISIT( $G, u$ )
  
```

DFS-VISIT(G, u)

```

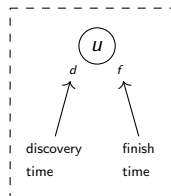
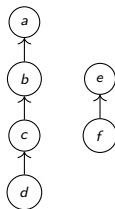
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



stack

$time = 11$

DFS
forest:





The DFS Algorithm

DFS(G)

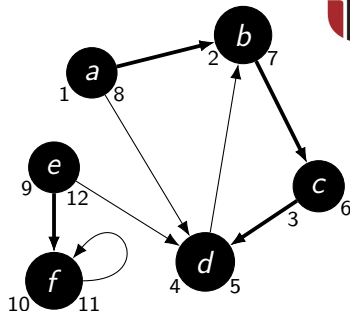
```

1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6    if  $u.color == WHITE$ 
7      DFS-VISIT( $G, u$ )
  
```

DFS-VISIT(G, u)

```

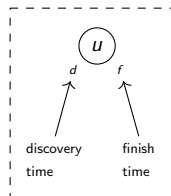
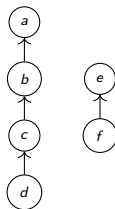
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



stack

$time = 12$

DFS
forest:



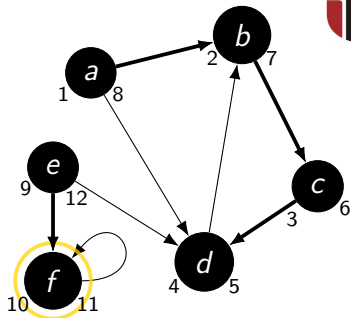


The DFS Algorithm

DFS(G)

```

1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6    if  $u.color == WHITE$ 
7      DFS-VISIT( $G, u$ )
  
```



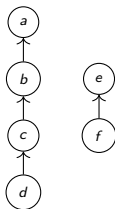
DFS-VISIT(G, u)

```

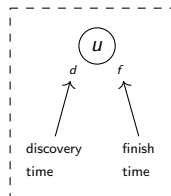
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```

stack

DFS
forest:



$time = 12$



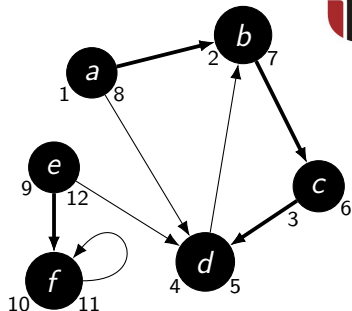


The DFS Algorithm

DFS(G)

```

1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
  
```



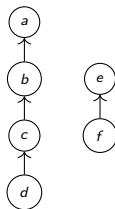
DFS-VISIT(G, u)

```

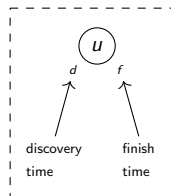
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```

stack

DFS
forest:



$time = 12$





DFS Algorithm - Complexity

DFS(G)

```
1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
```

DFS-VISIT(G, u)

```
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$ 
9   $time = time + 1$ 
10  $u.f = time$ 
```



DFS Algorithm - Complexity

DFS(G)

```

1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
  
```

$\rightarrow O(V)$

DFS-VISIT(G, u)

```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



DFS Algorithm - Complexity

DFS(G)

```

1  for each vertex  $u \in G.V$  }
2       $u.color = \text{WHITE}$    }
3       $u.\pi = \text{NIL}$          }
4   $time = 0$ 
5  for each vertex  $u \in G.V$  }
6      if  $u.color == \text{WHITE}$  }
7          DFS-VISIT( $G, u$ )    }
  
```

Diagram showing complexity analysis:

- The first loop (lines 1-3) is annotated with $O(V)$.
- The second loop (lines 5-7) is annotated with $O(V)$.

DFS-VISIT(G, u)

```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



DFS Algorithm - Complexity

DFS(G)

```

1  for each vertex  $u \in G.V$  }
2       $u.color = \text{WHITE}$    }
3       $u.\pi = \text{NIL}$          }
4   $time = 0$ 
5  for each vertex  $u \in G.V$  }
6      if  $u.color == \text{WHITE}$  }
7          DFS-VISIT( $G, u$ )    }
  
```

Diagram annotations:

- Lines 1-3 are grouped by a brace and an arrow pointing to $O(V)$.
- Lines 5-7 are grouped by a brace and an arrow pointing to $O(V)$.

DFS-VISIT(G, u) $\longrightarrow$ called $|V|$ times

```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```



DFS Algorithm - Complexity

DFS(G)

```

1  for each vertex  $u \in G.V$  }
2       $u.color = WHITE$ 
3       $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$  }
6      if  $u.color == WHITE$  }
7          DFS-VISIT( $G, u$ )
  
```

Annotations for DFS(G):

- Lines 1-3: $O(V)$
- Lines 5-7: $O(V)$

DFS-VISIT(G, u) $\longrightarrow$ called $|V|$ times

```

1   $time = time + 1$  }
2   $u.d = time$ 
3   $u.color = GRAY$  }
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == WHITE$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = BLACK$  }
9   $time = time + 1$ 
10  $u.f = time$ 
  
```

Annotations for DFS-VISIT(G, u):

- Lines 1-3: $O(1)$
- Lines 8-10: $O(1)$



DFS Algorithm - Complexity

DFS(G)

```

1  for each vertex  $u \in G.V$  }
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$  }
6      if  $u.color == \text{WHITE}$  }
7          DFS-VISIT( $G, u$ )
  
```

Annotations for DFS(G):

- Lines 1-3: $O(V)$
- Lines 5-7: $O(V)$

DFS-VISIT(G, u) $\rightarrow$ called $|V|$ times

```

1   $time = time + 1$  }
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```

Annotations for DFS-VISIT(G, u):

- Lines 1-3: $O(1)$
- Lines 4-7: each neighbor for every vertex, so $|E|$ steps in total
- Lines 8-10: $O(1)$



DFS Algorithm - Complexity

DFS(G)

```

1  for each vertex  $u \in G.V$  }
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$  }
6      if  $u.color == \text{WHITE}$  }
7          DFS-VISIT( $G, u$ )
  
```

Annotations for DFS(G):

- Lines 1-3: $O(V)$
- Lines 5-7: $O(V)$

DFS-VISIT(G, u) $\rightarrow$ called $|V|$ times

```

1   $time = time + 1$  }
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$  }
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```

Annotations for DFS-VISIT(G, u):

- Lines 1-3: $O(1)$
- Lines 4-7: each neighbor for every vertex, so $|E|$ steps in total
- Line 8: $O(1)$
- Lines 9-10: $O(1)$



DFS Algorithm - Complexity

DFS(G)

```

1  for each vertex  $u \in G.V$ 
2       $u.color = WHITE$ 
3       $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == WHITE$ 
7          DFS-VISIT( $G, u$ )
  
```

Complexity: $O(V + E)$

$\rightarrow O(V)$

$\rightarrow O(V)$

DFS-VISIT(G, u) $\rightarrow$ called $|V|$ times

```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == WHITE$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```

$\rightarrow O(1)$

each neighbor
for every vertex,
so $|E|$ steps in total

$\rightarrow O(1)$

$\rightarrow O(1)$



Parenthesis theorem

For any graph $G = (V, E)$ and for any $u, v \in G.V$, exactly one of the three conditions hold:

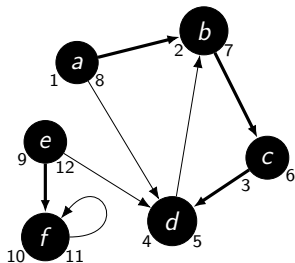
- $[u.d, u.f] \cap [v.d, v.f] = \emptyset$
- $[u.d, u.f] \subset [v.d, v.f]$
- $[v.d, v.f] \subset [u.d, u.f]$



Parenthesis theorem

For any graph $G = (V, E)$ and for any $u, v \in G.V$, exactly one of the three conditions hold:

- $[u.d, u.f] \cap [v.d, v.f] = \emptyset$
- $[u.d, u.f] \subset [v.d, v.f]$
- $[v.d, v.f] \subset [u.d, u.f]$

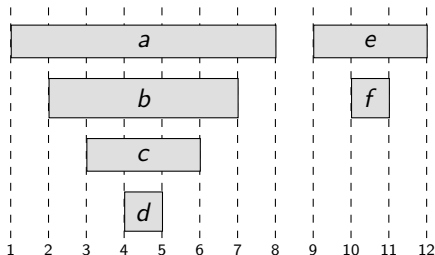
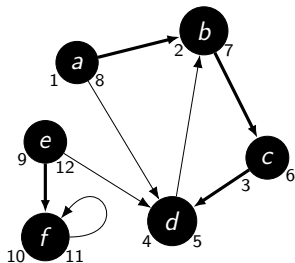




Parenthesis theorem

For any graph $G = (V, E)$ and for any $u, v \in G.V$, exactly one of the three conditions hold:

- $[u.d, u.f] \cap [v.d, v.f] = \emptyset$
- $[u.d, u.f] \subset [v.d, v.f]$
- $[v.d, v.f] \subset [u.d, u.f]$

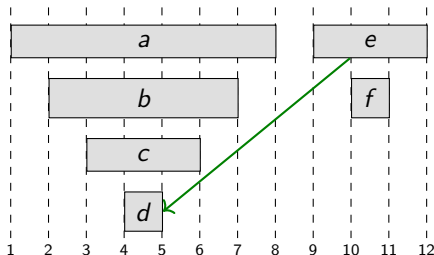
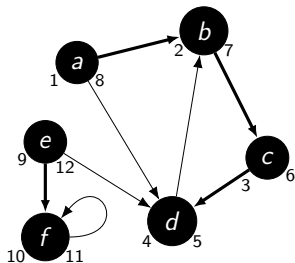




Parenthesis theorem

For any graph $G = (V, E)$ and for any $u, v \in G.V$, exactly one of the three conditions hold:

- $[u.d, u.f] \cap [v.d, v.f] = \emptyset$
neither u nor v is a descendant of the other
- $[u.d, u.f] \subset [v.d, v.f]$
- $[v.d, v.f] \subset [u.d, u.f]$

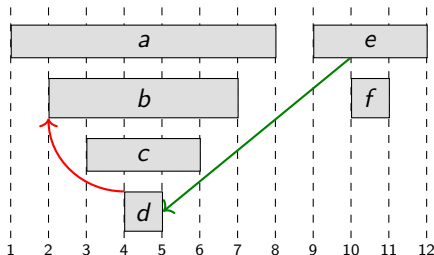
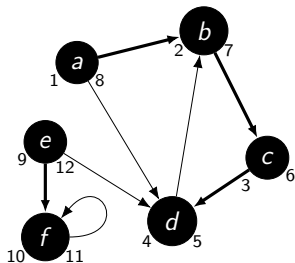




Parenthesis theorem

For any graph $G = (V, E)$ and for any $u, v \in G.V$, exactly one of the three conditions hold:

- $[u.d, u.f] \cap [v.d, v.f] = \emptyset$
neither u nor v is a descendant of the other
- $[u.d, u.f] \subset [v.d, v.f]$
 u is a descendant of v
- $[v.d, v.f] \subset [u.d, u.f]$

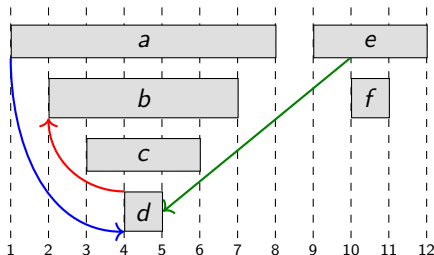
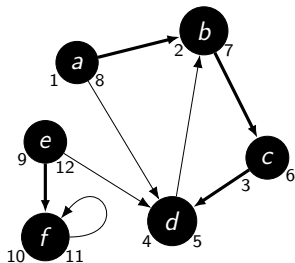




Parenthesis theorem

For any graph $G = (V, E)$ and for any $u, v \in G.V$, exactly one of the three conditions hold:

- $[u.d, u.f] \cap [v.d, v.f] = \emptyset$
neither u nor v is a descendant of the other
- $[u.d, u.f] \subset [v.d, v.f]$
 u is a descendant of v
- $[v.d, v.f] \subset [u.d, u.f]$
 v is a descendant of u





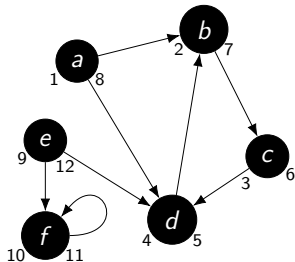
Edges characterization

- **Tree edges**
- **Back edges**
- **Forward edges**
- **Cross edges**



Edges characterization

- Tree edges
- Back edges
- Forward edges
- Cross edges





Edges characterization

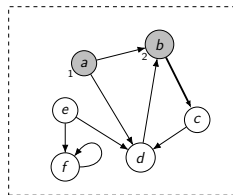
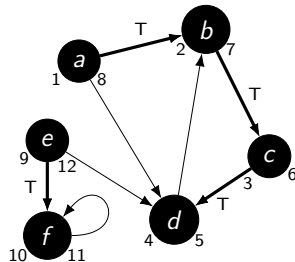
- **Tree edges**

- $u = \text{GRAY}; v = \text{WHITE}$
- $u.d < v.d < v.f < u.f$

- **Back edges**

- **Forward edges**

- **Cross edges**





Edges characterization

Tree edges

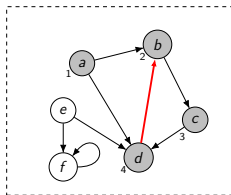
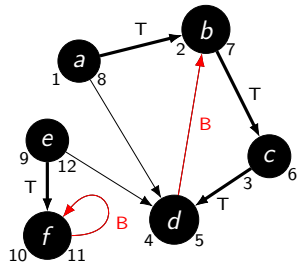
- $u = \text{GRAY}; v = \text{WHITE}$
- $u.d < v.d < v.f < u.f$

Back edges

- $u = \text{GRAY}; v = \text{GRAY}$
- $v.d < u.d < u.f < v.f$
- also self-loops

Forward edges

Cross edges





Edges characterization

Tree edges

- $u = \text{GRAY}; v = \text{WHITE}$
- $u.d < v.d < v.f < u.f$

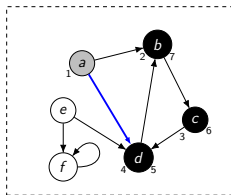
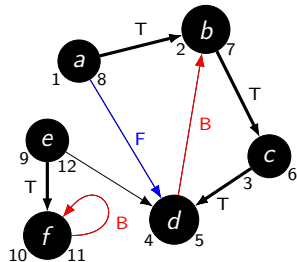
Back edges

- $u = \text{GRAY}; v = \text{GRAY}$
- $v.d < u.d < u.f < v.f$
- also self-loops

Forward edges

- $u = \text{GRAY}; v = \text{BLACK}$
- $u.d < v.d < v.f < u.f$

Cross edges





Edges characterization

Tree edges

- $u = \text{GRAY}; v = \text{WHITE}$
- $u.d < v.d < v.f < u.f$

Back edges

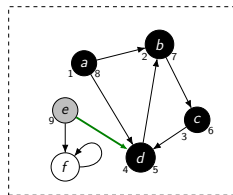
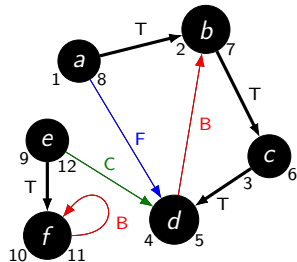
- $u = \text{GRAY}; v = \text{GRAY}$
- $v.d < u.d < u.f < v.f$
- also self-loops

Forward edges

- $u = \text{GRAY}; v = \text{BLACK}$
- $u.d < v.d < v.f < u.f$

Cross edges

- $u = \text{GRAY}; v = \text{BLACK}$
- $v.d < v.f < u.d < u.f$





Edges characterization

• Tree edges

- $u = \text{GRAY}; v = \text{WHITE}$
- $u.d < v.d < v.f < u.f$

• Back edges

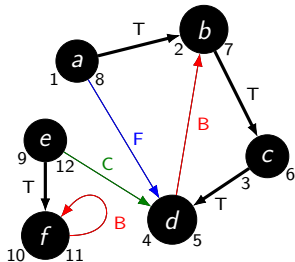
- $u = \text{GRAY}; v = \text{GRAY}$
- $v.d < u.d < u.f < v.f$
- also self-loops

• Forward edges

- $u = \text{GRAY}; v = \text{BLACK}$
- $u.d < v.d < v.f < u.f$

• Cross edges

- $u = \text{GRAY}; v = \text{BLACK}$
- $v.d < v.f < u.d < u.f$



Q: which edges produce cycles?



Edges characterization

• Tree edges

- $u = \text{GRAY}; v = \text{WHITE}$
- $u.d < v.d < v.f < u.f$

• Back edges [they produce cycles]

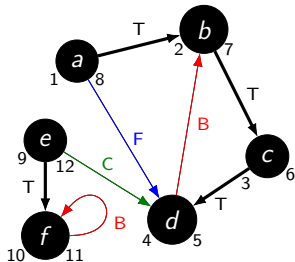
- $u = \text{GRAY}; v = \text{GRAY}$
- $v.d < u.d < u.f < v.f$
- also self-loops

• Forward edges

- $u = \text{GRAY}; v = \text{BLACK}$
- $u.d < v.d < v.f < u.f$

• Cross edges

- $u = \text{GRAY}; v = \text{BLACK}$
- $v.d < v.f < u.d < u.f$





Edges characterization - undirected graphs

- which types of edges can we encounter in undirected graphs?



Edges characterization - undirected graphs

- which types of edges can we encounter in undirected graphs?

Theorem

In a depth-first search of an undirected graph G , every edge of G is either a tree edge or a back edge.

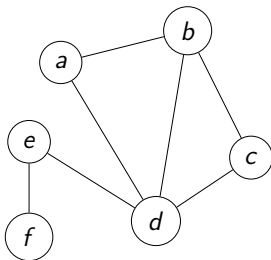


Edges characterization - undirected graphs

- which types of edges can we encounter in undirected graphs?

Theorem

In a depth-first search of an undirected graph G , every edge of G is either a tree edge or a back edge.



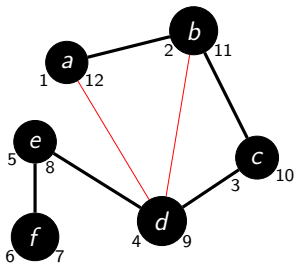


Edges characterization - undirected graphs

- which types of edges can we encounter in undirected graphs?

Theorem

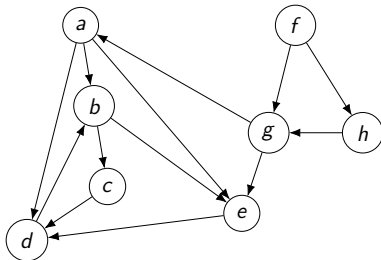
In a depth-first search of an undirected graph G , every edge of G is either a tree edge or a back edge.





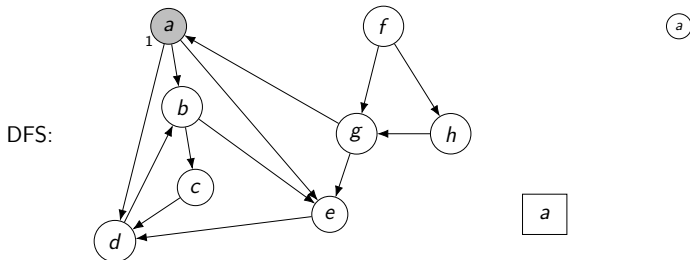
DFS vs. BFS example

DFS:



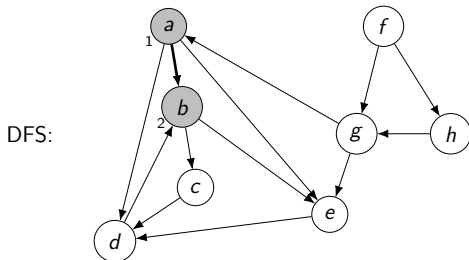


DFS vs. BFS example





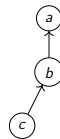
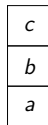
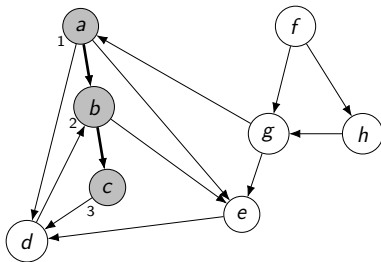
DFS vs. BFS example





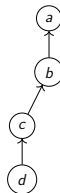
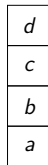
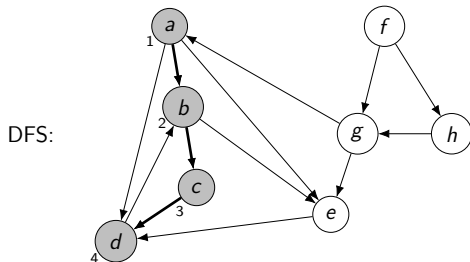
DFS vs. BFS example

DFS:



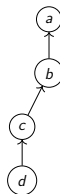
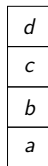
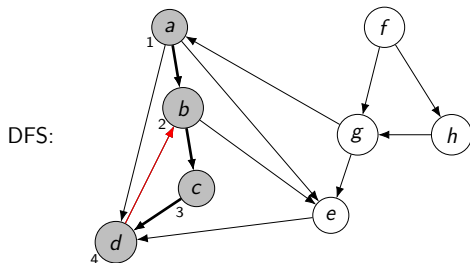


DFS vs. BFS example



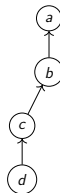
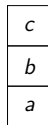
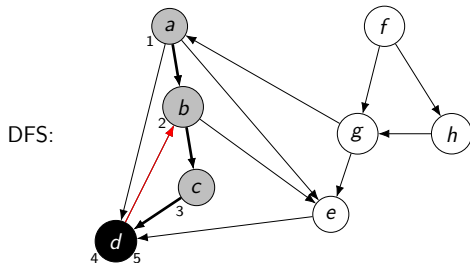


DFS vs. BFS example





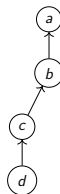
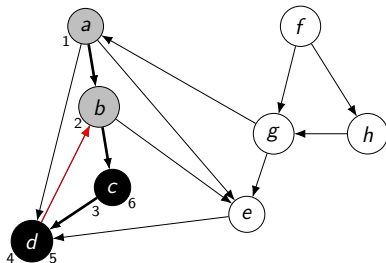
DFS vs. BFS example





DFS vs. BFS example

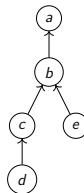
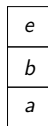
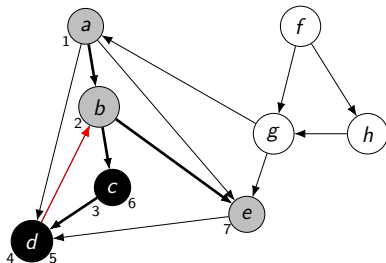
DFS:





DFS vs. BFS example

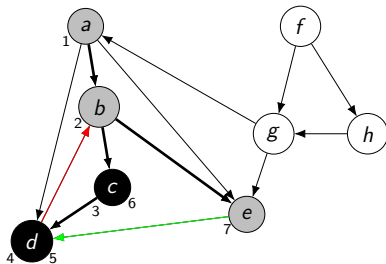
DFS:



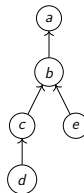


DFS vs. BFS example

DFS:

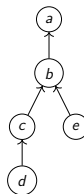
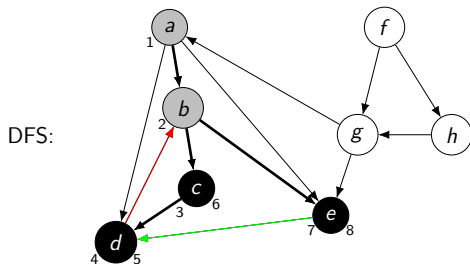


e
b
a



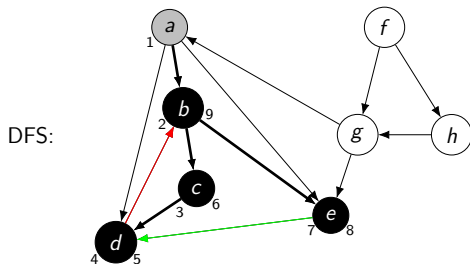


DFS vs. BFS example

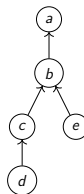




DFS vs. BFS example

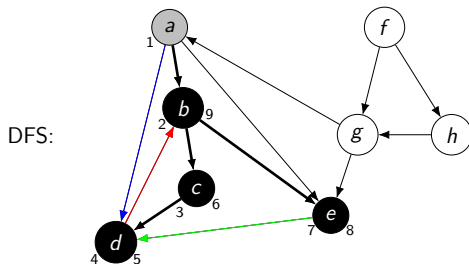


a

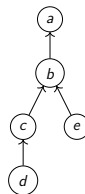




DFS vs. BFS example

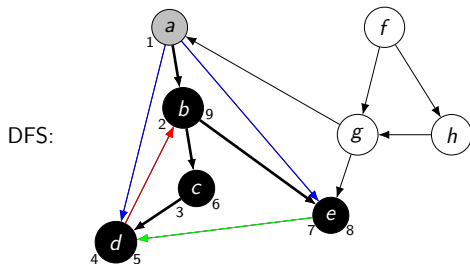


a

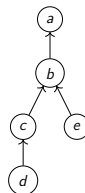




DFS vs. BFS example



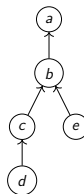
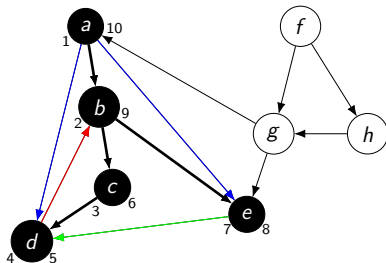
a





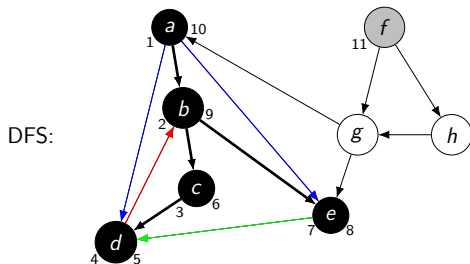
DFS vs. BFS example

DFS:

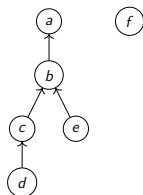




DFS vs. BFS example



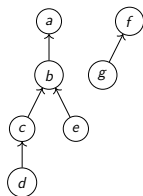
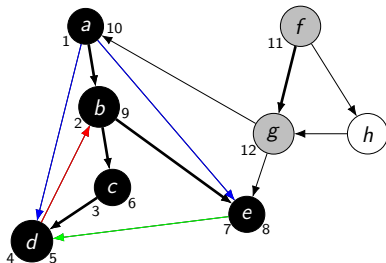
f





DFS vs. BFS example

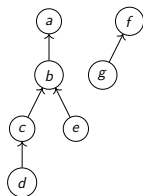
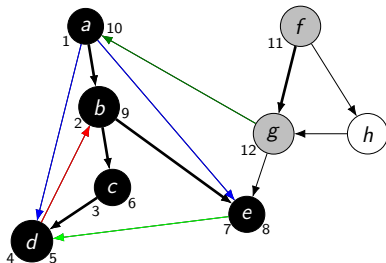
DFS:





DFS vs. BFS example

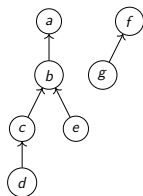
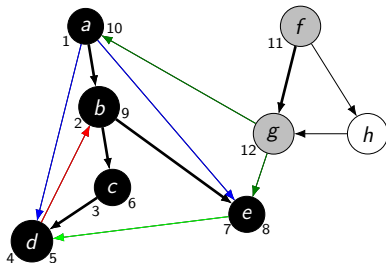
DFS:





DFS vs. BFS example

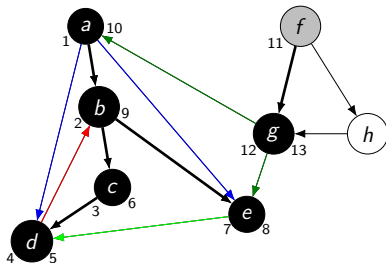
DFS:



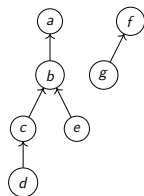


DFS vs. BFS example

DFS:



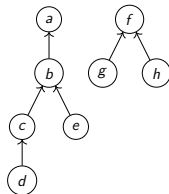
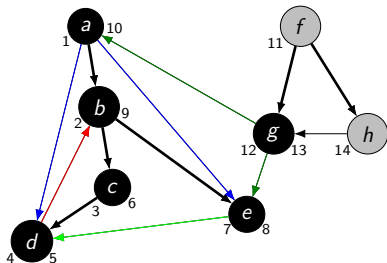
f





DFS vs. BFS example

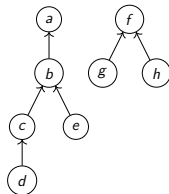
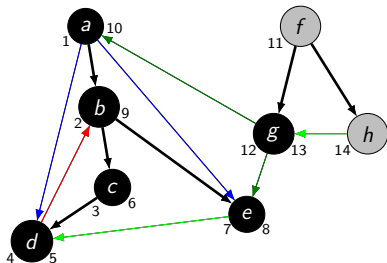
DFS:





DFS vs. BFS example

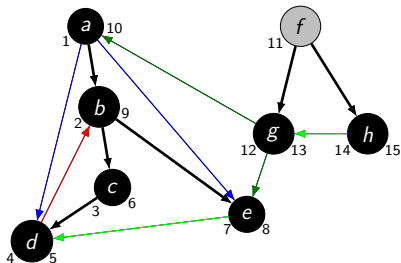
DFS:



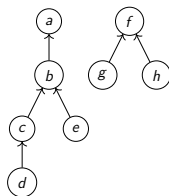


DFS vs. BFS example

DFS:



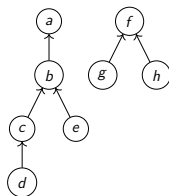
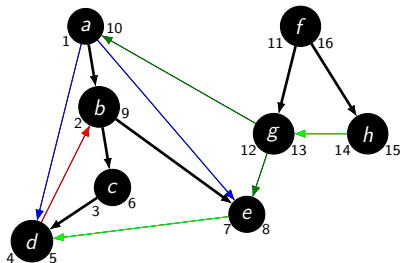
f





DFS vs. BFS example

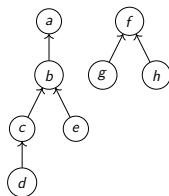
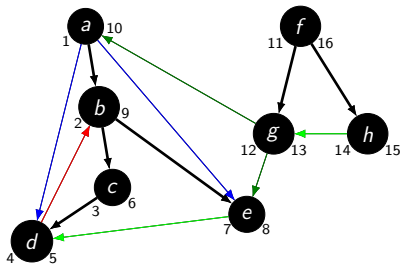
DFS:



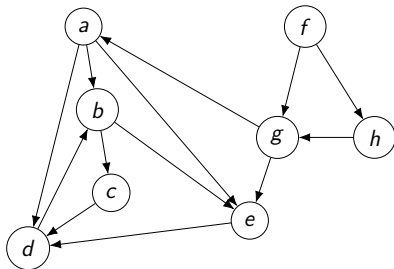


DFS vs. BFS example

DFS:



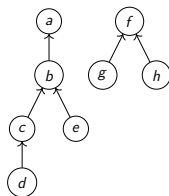
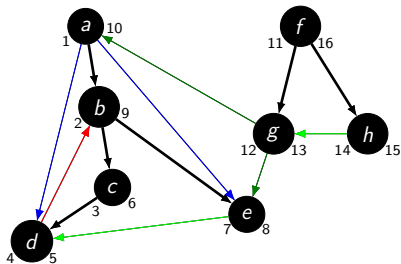
BFS:



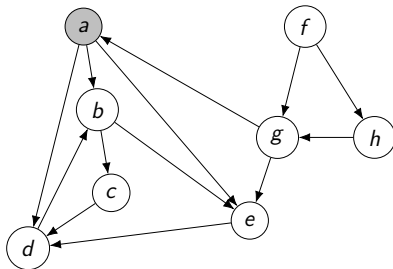


DFS vs. BFS example

DFS:



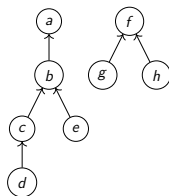
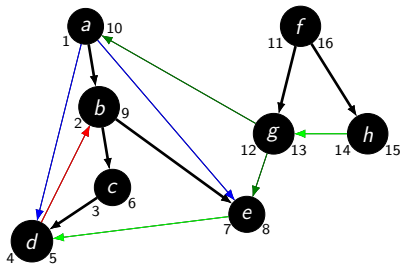
BFS:



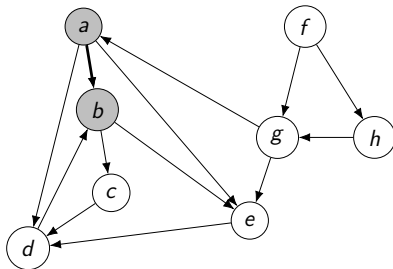


DFS vs. BFS example

DFS:



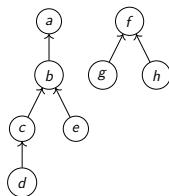
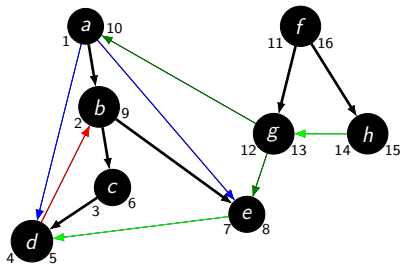
BFS:



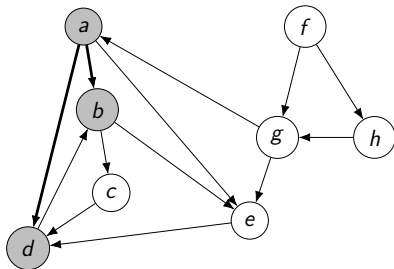


DFS vs. BFS example

DFS:



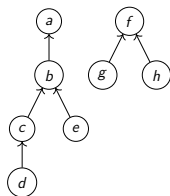
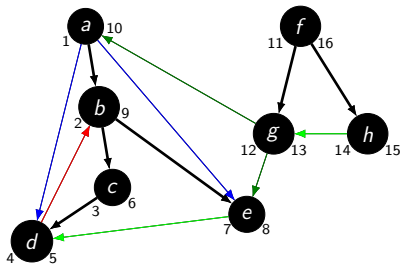
BFS:



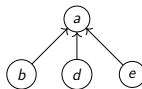
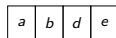
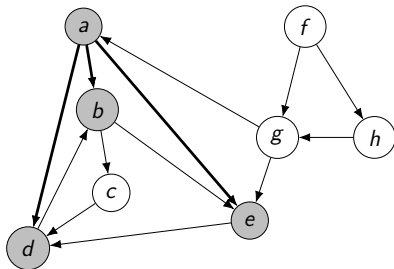


DFS vs. BFS example

DFS:



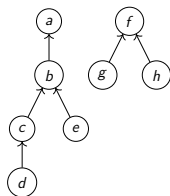
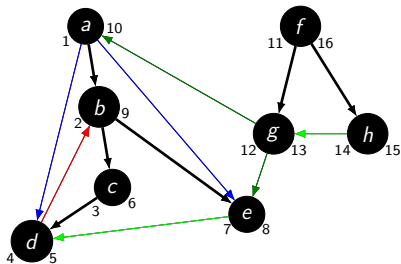
BFS:



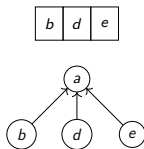
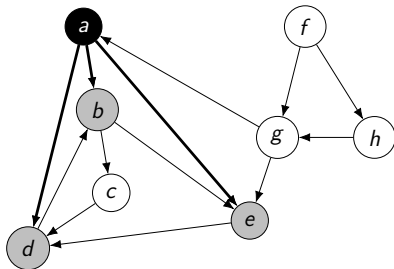


DFS vs. BFS example

DFS:



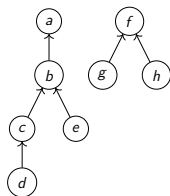
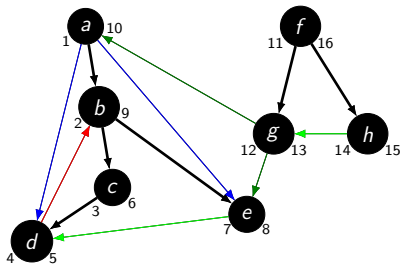
BFS:



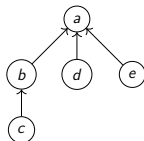
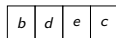
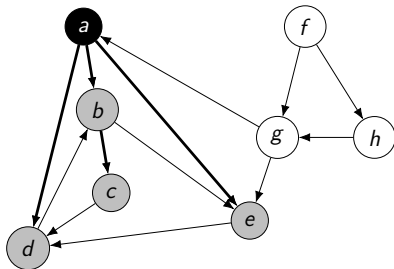


DFS vs. BFS example

DFS:



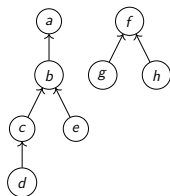
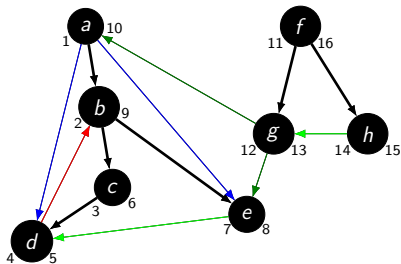
BFS:



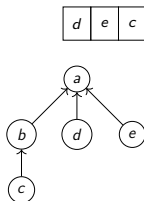
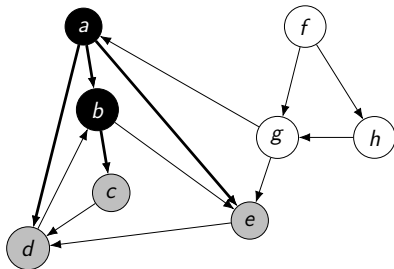


DFS vs. BFS example

DFS:



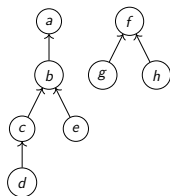
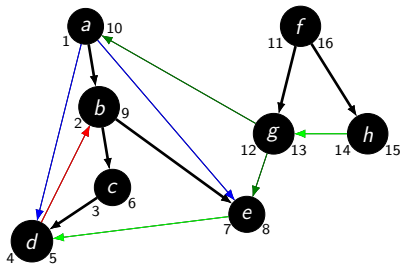
BFS:



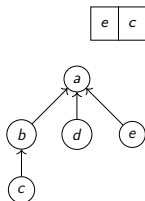
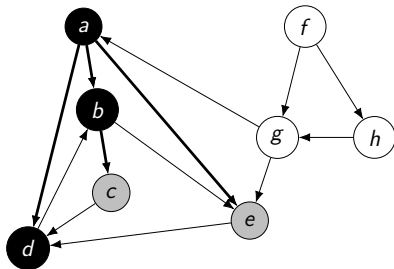


DFS vs. BFS example

DFS:



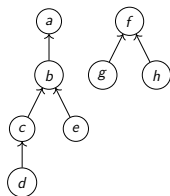
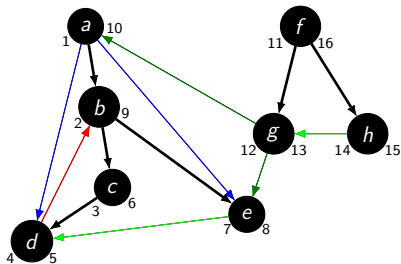
BFS:



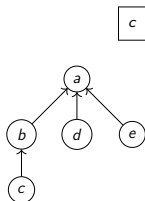
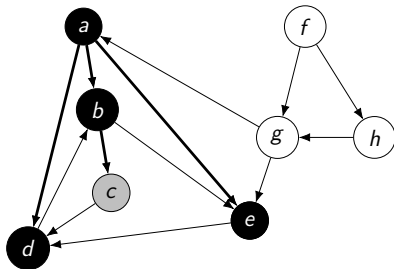


DFS vs. BFS example

DFS:



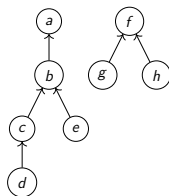
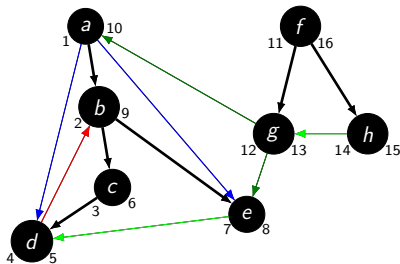
BFS:



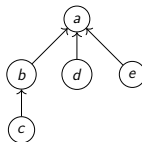
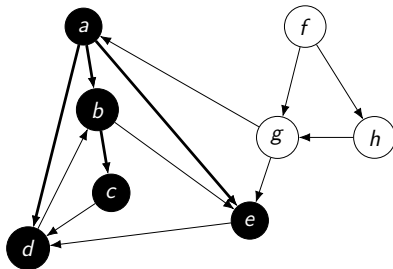


DFS vs. BFS example

DFS:



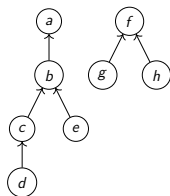
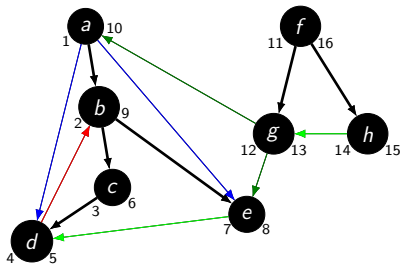
BFS:



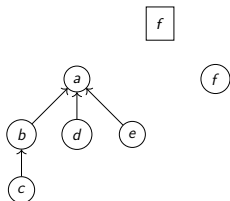
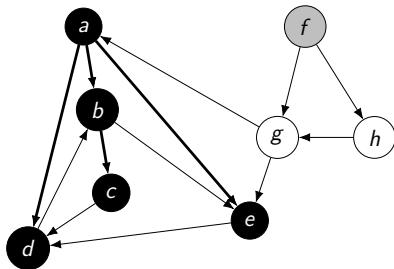


DFS vs. BFS example

DFS:



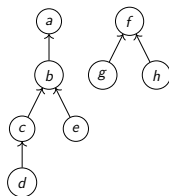
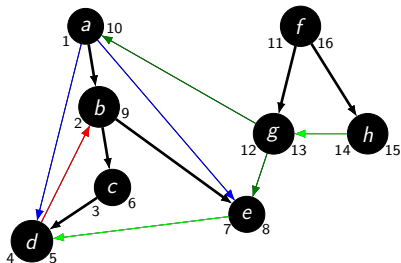
BFS:



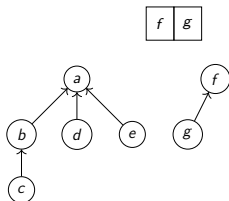
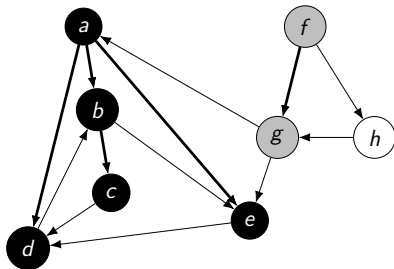


DFS vs. BFS example

DFS:



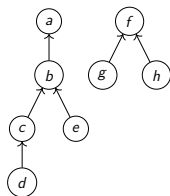
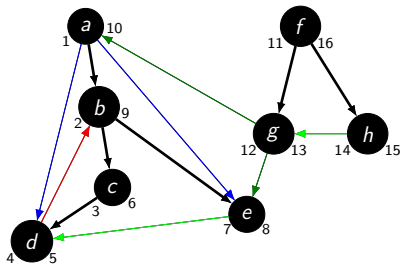
BFS:



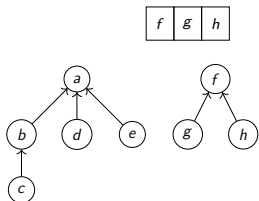
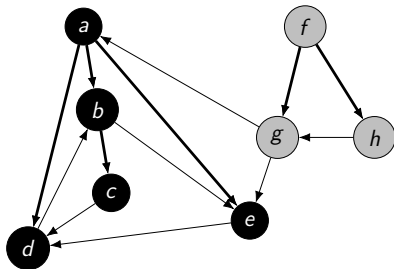


DFS vs. BFS example

DFS:



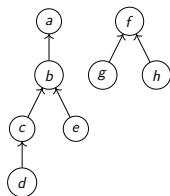
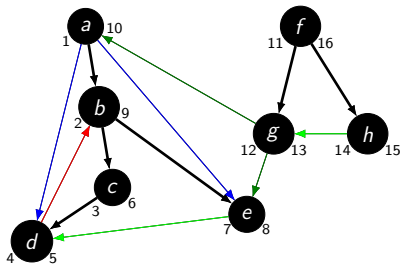
BFS:



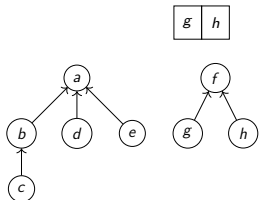
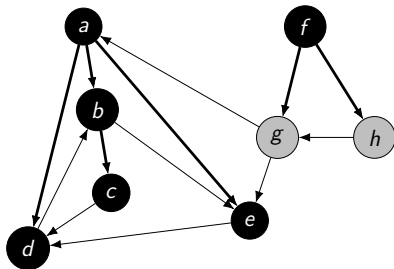


DFS vs. BFS example

DFS:



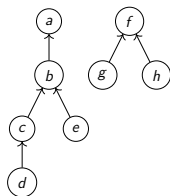
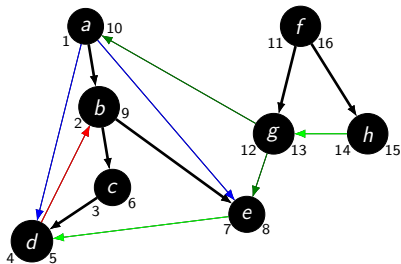
BFS:



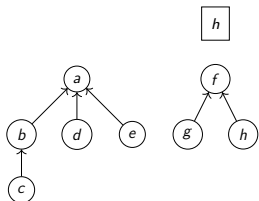
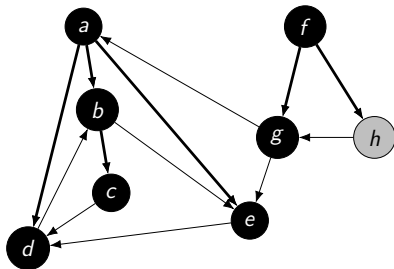


DFS vs. BFS example

DFS:



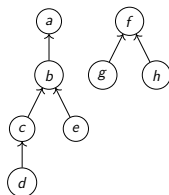
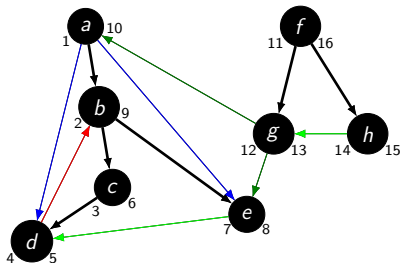
BFS:



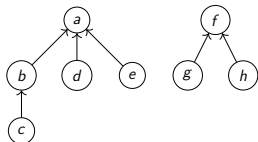
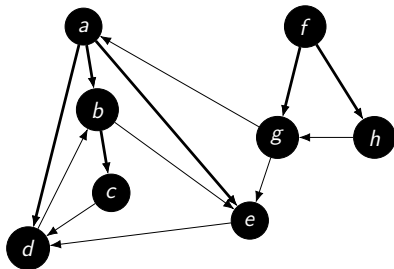


DFS vs. BFS example

DFS:



BFS:





Agenda

- 1 Depth First Search (DFS)
- 2 Connected components
 - Connected components
 - Strongly connected components



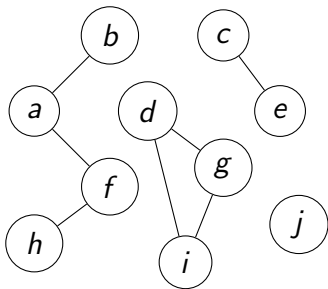
Connected components

- An undirected graph $G = (V, E)$ is **connected** if $\forall u, v \in V$ there is a path $u \rightsquigarrow v$ (u and v are reachable from one another).
- A **connected component** is a maximal set of vertices $C \subseteq V$ such that C is connected.



Connected components

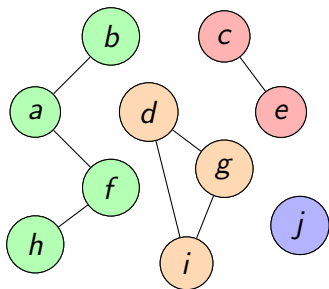
- An undirected graph $G = (V, E)$ is **connected** if $\forall u, v \in V$ there is a path $u \rightsquigarrow v$ (u and v are reachable from one another).
- A **connected component** is a maximal set of vertices $C \subseteq V$ such that C is connected.





Connected components

- An undirected graph $G = (V, E)$ is **connected** if $\forall u, v \in V$ there is a path $u \rightsquigarrow v$ (u and v are reachable from one another).
- A **connected component** is a maximal set of vertices $C \subseteq V$ such that C is connected.

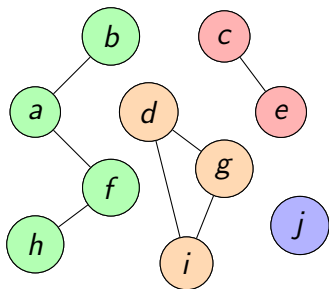


- $\{a, b, f, h\}$, $\{c, e\}$, $\{d, g, i\}$ and $\{j\}$ are connected components



Connected components

- An undirected graph $G = (V, E)$ is **connected** if $\forall u, v \in V$ there is a path $u \rightsquigarrow v$ (u and v are reachable from one another).
- A **connected component** is a maximal set of vertices $C \subseteq V$ such that C is connected.



- $\{a, b, f, h\}$, $\{c, e\}$, $\{d, g, i\}$ and $\{j\}$ are connected components
- $\{a, b, c\}$ is not a connected component because $\nexists a \rightsquigarrow c$
- $\{a, b, h\}$ is not a connected component because it is not maximal



Finding CC using Disjoint Sets

CONNECTED-COMPONENTS(G)

```
1  for each vertex  $v \in G.V$ 
2      MAKE-SET( $v$ )
3  for each edge  $(u, v) \in G.E$ 
4      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
5          UNION( $u, v$ )
```

SAME-COMPONENT(u, v)

```
1  if FIND-SET( $u$ ) == FIND-SET( $v$ )
2      return TRUE
3  else return FALSE
```



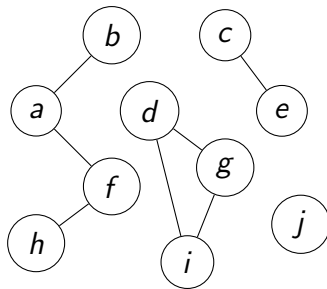
Finding CC using Disjoint Sets

CONNECTED-COMPONENTS(G)

```
1  for each vertex  $v \in G.V$ 
2      MAKE-SET( $v$ )
3  for each edge  $(u, v) \in G.E$ 
4      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
5          UNION( $u, v$ )
```

SAME-COMPONENT(u, v)

```
1  if FIND-SET( $u$ ) == FIND-SET( $v$ )
2      return TRUE
3  else return FALSE
```





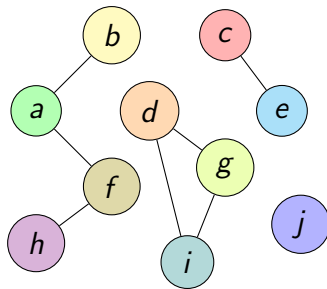
Finding CC using Disjoint Sets

CONNECTED-COMPONENTS(G)

```
1  for each vertex  $v \in G.V$ 
2      MAKE-SET( $v$ )
3  for each edge  $(u, v) \in G.E$ 
4      if  $\text{FIND-SET}(u) \neq \text{FIND-SET}(v)$ 
5          UNION( $u, v$ )
```

SAME-COMPONENT(u, v)

```
1  if  $\text{FIND-SET}(u) == \text{FIND-SET}(v)$ 
2      return TRUE
3  else return FALSE
```





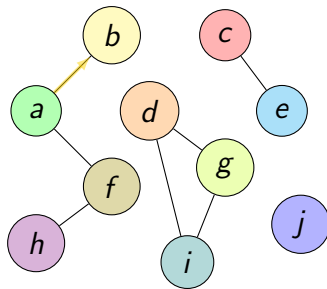
Finding CC using Disjoint Sets

CONNECTED-COMPONENTS(G)

```
1  for each vertex  $v \in G.V$ 
2      MAKE-SET( $v$ )
3  for each edge  $(u, v) \in G.E$ 
4      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
5          UNION( $u, v$ )
```

SAME-COMPONENT(u, v)

```
1  if FIND-SET( $u$ ) == FIND-SET( $v$ )
2      return TRUE
3  else return FALSE
```





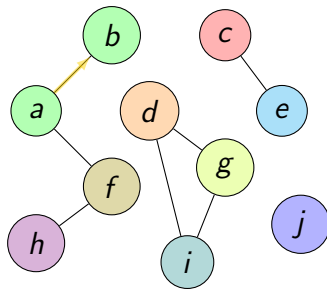
Finding CC using Disjoint Sets

CONNECTED-COMPONENTS(G)

```
1  for each vertex  $v \in G.V$ 
2    MAKE-SET( $v$ )
3  for each edge  $(u, v) \in G.E$ 
4    if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
5      UNION( $u, v$ )
```

SAME-COMPONENT(u, v)

```
1  if FIND-SET( $u$ ) == FIND-SET( $v$ )
2    return TRUE
3  else return FALSE
```





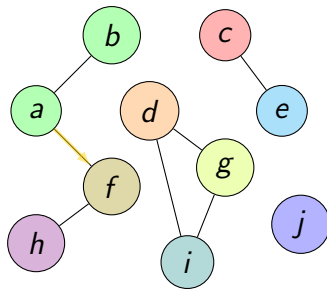
Finding CC using Disjoint Sets

CONNECTED-COMPONENTS(G)

```
1  for each vertex  $v \in G.V$ 
2      MAKE-SET( $v$ )
3  for each edge  $(u, v) \in G.E$ 
4      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
5          UNION( $u, v$ )
```

SAME-COMPONENT(u, v)

```
1  if FIND-SET( $u$ ) == FIND-SET( $v$ )
2      return TRUE
3  else return FALSE
```





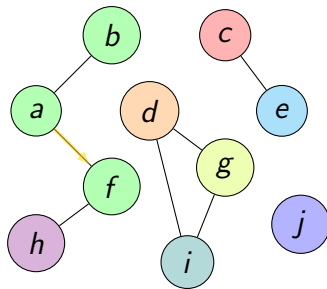
Finding CC using Disjoint Sets

CONNECTED-COMPONENTS(G)

```
1  for each vertex  $v \in G.V$ 
2    MAKE-SET( $v$ )
3  for each edge  $(u, v) \in G.E$ 
4    if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
5      UNION( $u, v$ )
```

SAME-COMPONENT(u, v)

```
1  if FIND-SET( $u$ ) == FIND-SET( $v$ )
2    return TRUE
3  else return FALSE
```





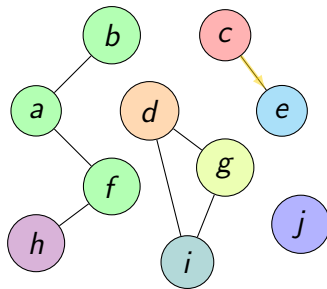
Finding CC using Disjoint Sets

CONNECTED-COMPONENTS(G)

```
1  for each vertex  $v \in G.V$ 
2      MAKE-SET( $v$ )
3  for each edge  $(u, v) \in G.E$ 
4      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
5          UNION( $u, v$ )
```

SAME-COMPONENT(u, v)

```
1  if FIND-SET( $u$ ) == FIND-SET( $v$ )
2      return TRUE
3  else return FALSE
```





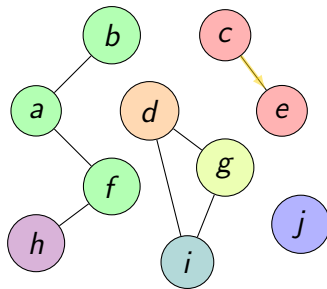
Finding CC using Disjoint Sets

CONNECTED-COMPONENTS(G)

```
1  for each vertex  $v \in G.V$ 
2      MAKE-SET( $v$ )
3  for each edge  $(u, v) \in G.E$ 
4      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
5          UNION( $u, v$ )
```

SAME-COMPONENT(u, v)

```
1  if FIND-SET( $u$ ) == FIND-SET( $v$ )
2      return TRUE
3  else return FALSE
```





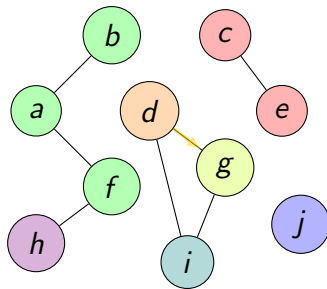
Finding CC using Disjoint Sets

CONNECTED-COMPONENTS(G)

```
1  for each vertex  $v \in G.V$ 
2      MAKE-SET( $v$ )
3  for each edge  $(u, v) \in G.E$ 
4      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
5          UNION( $u, v$ )
```

SAME-COMPONENT(u, v)

```
1  if FIND-SET( $u$ ) == FIND-SET( $v$ )
2      return TRUE
3  else return FALSE
```





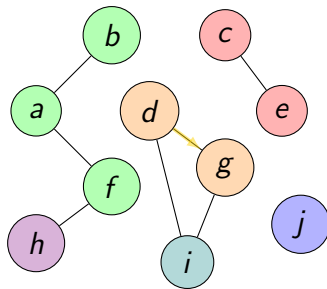
Finding CC using Disjoint Sets

CONNECTED-COMPONENTS(G)

```
1  for each vertex  $v \in G.V$ 
2    MAKE-SET( $v$ )
3  for each edge  $(u, v) \in G.E$ 
4    if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
5      UNION( $u, v$ )
```

SAME-COMPONENT(u, v)

```
1  if FIND-SET( $u$ ) == FIND-SET( $v$ )
2    return TRUE
3  else return FALSE
```





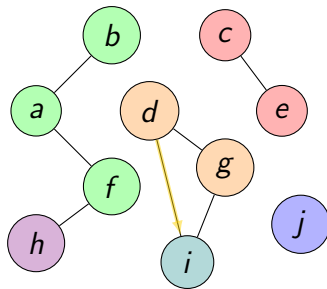
Finding CC using Disjoint Sets

CONNECTED-COMPONENTS(G)

```
1  for each vertex  $v \in G.V$ 
2      MAKE-SET( $v$ )
3  for each edge  $(u, v) \in G.E$ 
4      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
5          UNION( $u, v$ )
```

SAME-COMPONENT(u, v)

```
1  if FIND-SET( $u$ ) == FIND-SET( $v$ )
2      return TRUE
3  else return FALSE
```





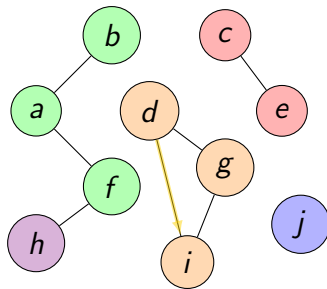
Finding CC using Disjoint Sets

CONNECTED-COMPONENTS(G)

```
1  for each vertex  $v \in G.V$ 
2    MAKE-SET( $v$ )
3  for each edge  $(u, v) \in G.E$ 
4    if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
5      UNION( $u, v$ )
```

SAME-COMPONENT(u, v)

```
1  if FIND-SET( $u$ ) == FIND-SET( $v$ )
2    return TRUE
3  else return FALSE
```





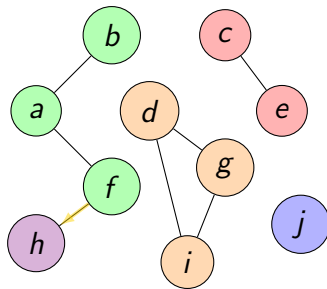
Finding CC using Disjoint Sets

CONNECTED-COMPONENTS(G)

```
1  for each vertex  $v \in G.V$ 
2    MAKE-SET( $v$ )
3  for each edge  $(u, v) \in G.E$ 
4    if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
5      UNION( $u, v$ )
```

SAME-COMPONENT(u, v)

```
1  if FIND-SET( $u$ ) == FIND-SET( $v$ )
2    return TRUE
3  else return FALSE
```





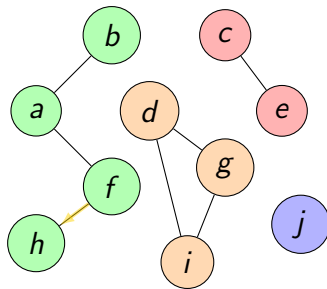
Finding CC using Disjoint Sets

CONNECTED-COMPONENTS(G)

```
1  for each vertex  $v \in G.V$ 
2    MAKE-SET( $v$ )
3  for each edge  $(u, v) \in G.E$ 
4    if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
5      UNION( $u, v$ )
```

SAME-COMPONENT(u, v)

```
1  if FIND-SET( $u$ ) == FIND-SET( $v$ )
2    return TRUE
3  else return FALSE
```





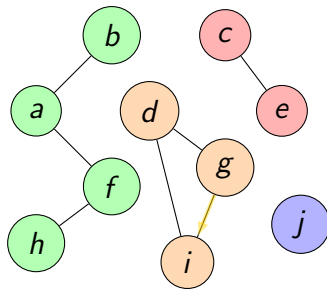
Finding CC using Disjoint Sets

CONNECTED-COMPONENTS(G)

```
1  for each vertex  $v \in G.V$ 
2      MAKE-SET( $v$ )
3  for each edge  $(u, v) \in G.E$ 
4      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
5          UNION( $u, v$ )
```

SAME-COMPONENT(u, v)

```
1  if FIND-SET( $u$ ) == FIND-SET( $v$ )
2      return TRUE
3  else return FALSE
```





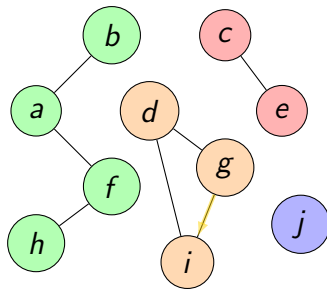
Finding CC using Disjoint Sets

CONNECTED-COMPONENTS(G)

```
1  for each vertex  $v \in G.V$ 
2      MAKE-SET( $v$ )
3  for each edge  $(u, v) \in G.E$ 
4      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
5          UNION( $u, v$ )
```

SAME-COMPONENT(u, v)

```
1  if FIND-SET( $u$ ) == FIND-SET( $v$ )
2      return TRUE
3  else return FALSE
```





Finding connected components using DFS

- use DFS as a skeleton

DFS-CC(G)

```

1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4
5   $time = 0$ 
6
7  for each vertex  $u \in G.V$ 
8      if  $u.color == \text{WHITE}$ 
9
10         DFS-VISIT-CC( $G, u$ )
  
```

DFS-VISIT-CC(G, u)

```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT-CC( $G, v$ )
8   $u.color = \text{BLACK}$ 
9
10  $time = time + 1$ 
11  $u.f = time$ 
  
```



Finding connected components using DFS

- use DFS as a skeleton
- add the *comp* attribute for each node
- the current *comp* value increases when a new DFS tree is started

DFS-CC(*G*)

```

1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4       $u.comp = 0$ 
5   $time = 0$ 
6   $comp = 0$ 
7  for each vertex  $u \in G.V$ 
8      if  $u.color == \text{WHITE}$ 
9           $comp = comp + 1$ 
10         DFS-VISIT-CC( $G, u, comp$ )
  
```

DFS-VISIT-CC($G, u, comp$)

```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT-CC( $G, v, comp$ )
8   $u.color = \text{BLACK}$ 
9   $u.comp = comp$ 
10  $time = time + 1$ 
11  $u.f = time$ 
  
```



Finding connected components using DFS

- use DFS as a skeleton
- add the *comp* attribute for each node
- the current *comp* value increases when a new DFS tree is started
- remove unwanted attributes like π , d , f

DFS-CC(G)

```

1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4       $u.comp = 0$ 
5   $time = 0$ 
6   $comp = 0$ 
7  for each vertex  $u \in G.V$ 
8      if  $u.color == \text{WHITE}$ 
9           $comp = comp + 1$ 
10         DFS-VISIT-CC( $G, u, comp$ )

```

DFS-VISIT-CC($G, u, comp$)

```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT-CC( $G, v, comp$ )
8   $u.color = \text{BLACK}$ 
9   $u.comp = comp$ 
10  $time = time + 1$ 
11  $u.f = time$ 

```



Finding connected components using DFS

- use DFS as a skeleton
- add the *comp* attribute for each node
- the current *comp* value increases when a new DFS tree is started
- remove unwanted attributes like π , d , f
- the BFS algorithm can also be used instead of DFS

DFS-CC(G)

```

1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4       $u.comp = 0$ 
5   $time = 0$ 
6   $comp = 0$ 
7  for each vertex  $u \in G.V$ 
8      if  $u.color == \text{WHITE}$ 
9           $comp = comp + 1$ 
10         DFS-VISIT-CC( $G, u, comp$ )
  
```

DFS-VISIT-CC($G, u, comp$)

```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT-CC( $G, v, comp$ )
8   $u.color = \text{BLACK}$ 
9   $u.comp = comp$ 
10  $time = time + 1$ 
11  $u.f = time$ 
  
```



Performance - Disjoint Sets vs. DFS/BFS

- Disjoint Sets approach
 - for each vertex we call MAKE-SET
 - for each edge we call FIND-SET and UNION



Performance - Disjoint Sets vs. DFS/BFS

- Disjoint Sets approach
 - for each vertex we call MAKE-SET
 - for each edge we call FIND-SET and UNION
 - $O(V + E \cdot \alpha(V))$



Performance - Disjoint Sets vs. DFS/BFS

- Disjoint Sets approach
 - for each vertex we call MAKE-SET
 - for each edge we call FIND-SET and UNION
 - $O(V + E \cdot \alpha(V))$
- DFS/BFS



Performance - Disjoint Sets vs. DFS/BFS

- Disjoint Sets approach
 - for each vertex we call MAKE-SET
 - for each edge we call FIND-SET and UNION
 - $O(V + E \cdot \alpha(V))$
- DFS/BFS
 - $O(V + E)$



Performance - Disjoint Sets vs. DFS/BFS

- Disjoint Sets approach
 - for each vertex we call MAKE-SET
 - for each edge we call FIND-SET and UNION
 - $O(V + E \cdot \alpha(V))$
- DFS/BFS
 - $O(V + E)$
- normally, the DFS/BFS approach is faster



Performance - Disjoint Sets vs. DFS/BFS

- Disjoint Sets approach
 - for each vertex we call MAKE-SET
 - for each edge we call FIND-SET and UNION
 - $O(V + E \cdot \alpha(V))$
- DFS/BFS
 - $O(V + E)$
- normally, the DFS/BFS approach is faster
- there are custom scenarios where the Disjoint Sets approach is preferred



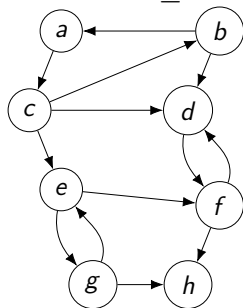
Strongly connected components

- A directed graph $G = (V, E)$ is **strongly connected** if $\forall u, v \in V$ there is a path $u \rightsquigarrow v$ and a path $v \rightsquigarrow u$ (u and v are reachable from one another).
- A **strongly connected component (SCC)** is a maximal set of vertices $C \subseteq V$ such that C is strongly connected.



Strongly connected components

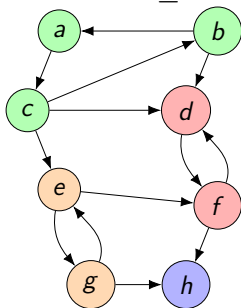
- A directed graph $G = (V, E)$ is **strongly connected** if $\forall u, v \in V$ there is a path $u \rightsquigarrow v$ and a path $v \rightsquigarrow u$ (u and v are reachable from one another).
- A **strongly connected component (SCC)** is a maximal set of vertices $C \subseteq V$ such that C is strongly connected.





Strongly connected components

- A directed graph $G = (V, E)$ is **strongly connected** if $\forall u, v \in V$ there is a path $u \rightsquigarrow v$ and a path $v \rightsquigarrow u$ (u and v are reachable from one another).
- A **strongly connected component (SCC)** is a maximal set of vertices $C \subseteq V$ such that C is strongly connected.

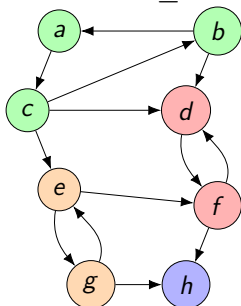


- $\{a, b, c\}$, $\{d, f\}$, $\{e, g\}$ and $\{h\}$ are SCC



Strongly connected components

- A directed graph $G = (V, E)$ is **strongly connected** if $\forall u, v \in V$ there is a path $u \rightsquigarrow v$ and a path $v \rightsquigarrow u$ (u and v are reachable from one another).
- A **strongly connected component (SCC)** is a maximal set of vertices $C \subseteq V$ such that C is strongly connected.



- $\{a, b, c\}$, $\{d, f\}$, $\{e, g\}$ and $\{h\}$ are SCC
- $\{a, b, c, d\}$ is not an SCC because $\nexists d \rightsquigarrow a$ (although $\exists a \rightsquigarrow d$)



Kosaraju's algorithm

STRONGLY-CONNECTED-COMPONENTS(G)

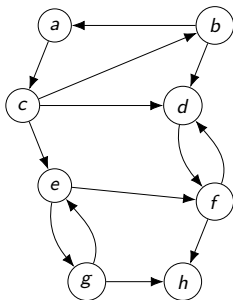
- 1 call DFS(G) to compute finishing times $u.f$, for each $u \in G.V$
- 2 compute G^T by reversing each edge direction
- 3 call DFS(G^T), but in the main loop consider the vertices in order of decreasing $u.f$ (as computed in line 1)
- 4 output the vertices of each tree in the DFS forest (line 3) as a separate SCC



Kosaraju's algorithm

STRONGLY-CONNECTED-COMPONENTS(G)

- 1 call DFS(G) to compute finishing times $u.f$, for each $u \in G.V$
- 2 compute G^T by reversing each edge direction
- 3 call DFS(G^T), but in the main loop consider the vertices in order of decreasing $u.f$ (as computed in line 1)
- 4 output the vertices of each tree in the DFS forest (line 3) as a separate SCC

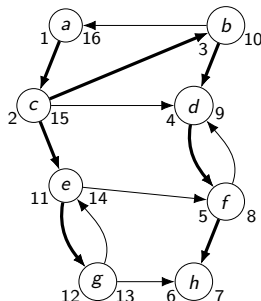




Kosaraju's algorithm

STRONGLY-CONNECTED-COMPONENTS(G)

- 1 call DFS(G) to compute finishing times $u.f$, for each $u \in G.V$
- 2 compute G^T by reversing each edge direction
- 3 call DFS(G^T), but in the main loop consider the vertices in order of decreasing $u.f$ (as computed in line 1)
- 4 output the vertices of each tree in the DFS forest (line 3) as a separate SCC

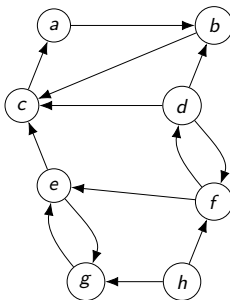
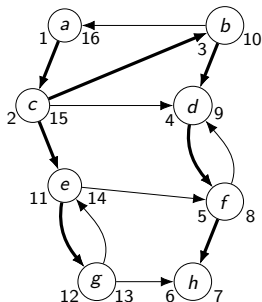




Kosaraju's algorithm

STRONGLY-CONNECTED-COMPONENTS(G)

- 1 call DFS(G) to compute finishing times $u.f$, for each $u \in G.V$
- 2 compute G^T by reversing each edge direction
- 3 call DFS(G^T), but in the main loop consider the vertices in order of decreasing $u.f$ (as computed in line 1)
- 4 output the vertices of each tree in the DFS forest (line 3) as a separate SCC

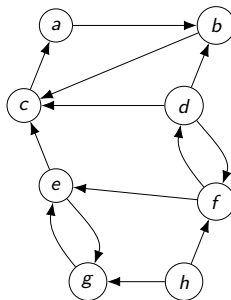
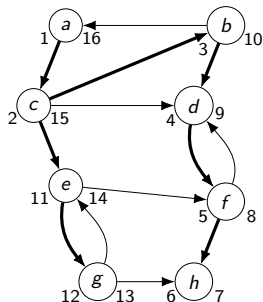




Kosaraju's algorithm

STRONGLY-CONNECTED-COMPONENTS(G)

- 1 call DFS(G) to compute finishing times $u.f$, for each $u \in G.V$
- 2 compute G^T by reversing each edge direction
- 3 call DFS(G^T), but in the main loop consider the vertices in order of decreasing $u.f$ (as computed in line 1)
- 4 output the vertices of each tree in the DFS forest (line 3) as a separate SCC



vertex order:

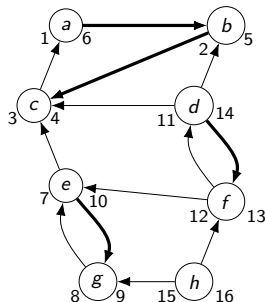
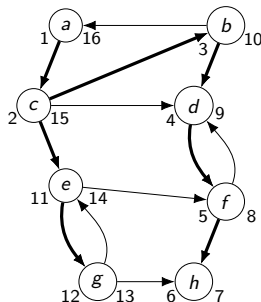
a, c, e, g, b, d, f, h



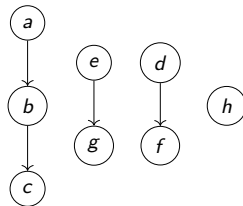
Kosaraju's algorithm

STRONGLY-CONNECTED-COMPONENTS(G)

- 1 call DFS(G) to compute finishing times $u.f$, for each $u \in G.V$
- 2 compute G^T by reversing each edge direction
- 3 call DFS(G^T), but in the main loop consider the vertices in order of decreasing $u.f$ (as computed in line 1)
- 4 output the vertices of each tree in the DFS forest (line 3) as a separate SCC



vertex order:
 a, c, e, g, b, d, f, h

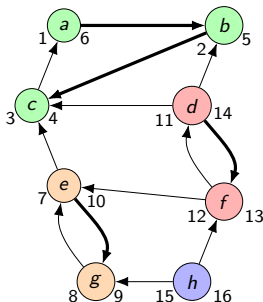
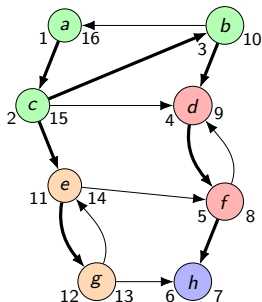




Kosaraju's algorithm

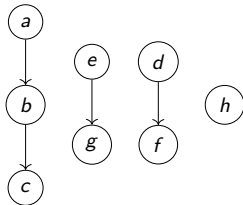
STRONGLY-CONNECTED-COMPONENTS(G)

- 1 call DFS(G) to compute finishing times $u.f$, for each $u \in G.V$
- 2 compute G^T by reversing each edge direction
- 3 call DFS(G^T), but in the main loop consider the vertices in order of decreasing $u.f$ (as computed in line 1)
- 4 output the vertices of each tree in the DFS forest (line 3) as a separate SCC



vertex order:

a, c, e, g, b, d, f, h





Kosaraju's algorithm - Complexity

STRONGLY-CONNECTED-COMPONENTS(G)

- 1 call DFS(G) to compute finishing times $u.f$, for each $u \in G.V$
- 2 compute G^T by reversing each edge direction
- 3 call DFS(G^T), but in the main loop consider the vertices in order of decreasing $u.f$ (as computed in line 1)
- 4 output the vertices of each tree in the DFS forest (line 3) as a separate SCC



Kosaraju's algorithm - Complexity

STRONGLY-CONNECTED-COMPONENTS(G)

- 1 call DFS(G) to compute finishing times $u.f$, for each $u \in G.V$
- 2 compute G^T by reversing each edge direction
- 3 call DFS(G^T), but in the main loop consider the vertices in order of decreasing $u.f$ (as computed in line 1)
- 4 output the vertices of each tree in the DFS forest (line 3) as a separate SCC

① DFS complexity: $O(V + E)$



Kosaraju's algorithm - Complexity

STRONGLY-CONNECTED-COMPONENTS(G)

- 1 call DFS(G) to compute finishing times $u.f$, for each $u \in G.V$
- 2 compute G^T by reversing each edge direction
- 3 call DFS(G^T), but in the main loop consider the vertices in order of decreasing $u.f$ (as computed in line 1)
- 4 output the vertices of each tree in the DFS forest (line 3) as a separate SCC

① DFS complexity: $O(V + E)$

② computing G^T : $O(E)$



Kosaraju's algorithm - Complexity

STRONGLY-CONNECTED-COMPONENTS(G)

- 1 call DFS(G) to compute finishing times $u.f$, for each $u \in G.V$
- 2 compute G^T by reversing each edge direction
- 3 call DFS(G^T), but in the main loop consider the vertices in order of decreasing $u.f$ (as computed in line 1)
- 4 output the vertices of each tree in the DFS forest (line 3) as a separate SCC

- ① DFS complexity: $O(V + E)$
- ② computing G^T : $O(E)$
- ③ DFS complexity: $O(V + E)$



Kosaraju's algorithm - Complexity

STRONGLY-CONNECTED-COMPONENTS(G)

- 1 call DFS(G) to compute finishing times $u.f$, for each $u \in G.V$
- 2 compute G^T by reversing each edge direction
- 3 call DFS(G^T), but in the main loop consider the vertices in order of decreasing $u.f$ (as computed in line 1)
- 4 output the vertices of each tree in the DFS forest (line 3) as a separate SCC

- ① DFS complexity: $O(V + E)$
- ② computing G^T : $O(E)$
- ③ DFS complexity: $O(V + E)$
- ④ traversing the DFS forest: $O(V)$



Kosaraju's algorithm - Complexity

STRONGLY-CONNECTED-COMPONENTS(G)

- 1 call DFS(G) to compute finishing times $u.f$, for each $u \in G.V$
- 2 compute G^T by reversing each edge direction
- 3 call DFS(G^T), but in the main loop consider the vertices in order of decreasing $u.f$ (as computed in line 1)
- 4 output the vertices of each tree in the DFS forest (line 3) as a separate SCC

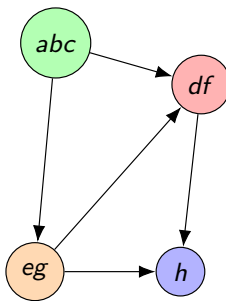
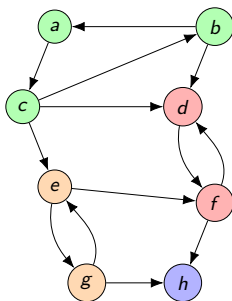
- ① DFS complexity: $O(V + E)$
- ② computing G^T : $O(E)$
- ③ DFS complexity: $O(V + E)$
- ④ traversing the DFS forest: $O(V)$

The algorithm complexity is $O(V + E)$.



Kosaraju's algorithm - Proof (1)

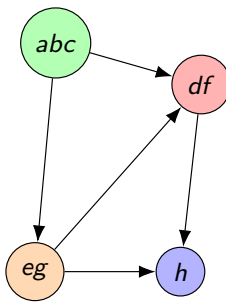
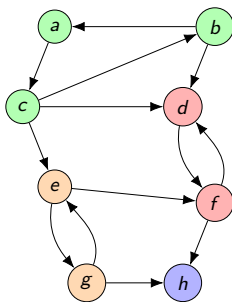
- the component graph $G^{SCC} = (V^{SCC}, E^{SCC})$
- the SCC are $C_1, C_2, \dots, C_k$
- $V^{SCC} = \{v_1, v_2, \dots, v_k\}$
- $(v_i, v_j) \in E^{SCC}$ if $\exists (x, y) \in E$, such that $x \in C_i, y \in C_j$





Kosaraju's algorithm - Proof (1)

- the component graph $G^{SCC} = (V^{SCC}, E^{SCC})$
- the SCC are $C_1, C_2, \dots, C_k$
- $V^{SCC} = \{v_1, v_2, \dots, v_k\}$
- $(v_i, v_j) \in E^{SCC}$ if $\exists (x, y) \in E$, such that $x \in C_i, y \in C_j$

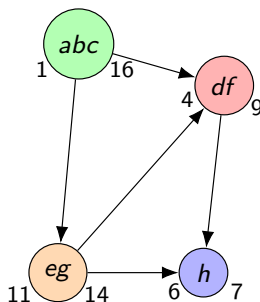
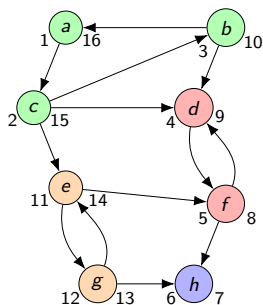


- G and G^T have the same SCC



Kosaraju's algorithm - Proof (1)

- the component graph $G^{SCC} = (V^{SCC}, E^{SCC})$
- the SCC are $C_1, C_2, \dots, C_k$
- $V^{SCC} = \{v_1, v_2, \dots, v_k\}$
- $(v_i, v_j) \in E^{SCC}$ if $\exists (x, y) \in E$, such that $x \in C_i, y \in C_j$



- G and G^T have the same SCC
- let C be a SCC of G
- $d(C) = \min_{u \in C} \{u.d\}$
- $f(C) = \max_{u \in C} \{u.f\}$



Kosaraju's algorithm - Proof (2)

Lemma 22.13

Let C and C' be distinct SCC in directed graph G . Let $u, v \in C$ and $u', v' \in C'$ and suppose G contains a path $u \rightsquigarrow u'$. Then G cannot contain a path $v' \rightsquigarrow v$.



Kosaraju's algorithm - Proof (2)

Lemma 22.13

Let C and C' be distinct SCC in directed graph G . Let $u, v \in C$ and $u', v' \in C'$ and suppose G contains a path $u \rightsquigarrow u'$. Then G cannot contain a path $v' \rightsquigarrow v$.

- by contradiction, if $\exists v' \rightsquigarrow v$, then
 - $\exists u \rightsquigarrow u' \rightsquigarrow v'$
 - $\exists v' \rightsquigarrow v \rightsquigarrow u$
- $\Rightarrow u$ and v' are reachable from each other
- $\Rightarrow C$ and C' are not distinct



Kosaraju's algorithm - Proof (3)

Lemma 22.14

Let C and C' be distinct SCC in directed graph $G = (V, E)$. Suppose that $(u, v) \in E$, with $u \in C$ and $v \in C'$. Then $f(C) > f(C')$.



Kosaraju's algorithm - Proof (3)

Lemma 22.14

Let C and C' be distinct SCC in directed graph $G = (V, E)$. Suppose that $(u, v) \in E$, with $u \in C$ and $v \in C'$. Then $f(C) > f(C')$.

- if $d(C) < d(C')$
 - Let x be the first vertex discovered in C . At the time $x.d$ all the vertices in C and C' are WHITE.
 - Any vertex in C and C' is a descendant of x in the DFS tree ($x \rightsquigarrow u \rightarrow v \rightsquigarrow y, \forall y \in C'$).
 - $\Rightarrow x.f = f(C) > f(C')$



Kosaraju's algorithm - Proof (3)

Lemma 22.14

Let C and C' be distinct SCC in directed graph $G = (V, E)$. Suppose that $(u, v) \in E$, with $u \in C$ and $v \in C'$. Then $f(C) > f(C')$.

- if $d(C) < d(C')$
 - Let x be the first vertex discovered in C . At the time $x.d$ all the vertices in C and C' are WHITE.
 - Any vertex in C and C' is a descendant of x in the DFS tree ($x \rightsquigarrow u \rightarrow v \rightsquigarrow y, \forall y \in C'$).
 - $\Rightarrow x.f = f(C) > f(C')$
- if $d(C) > d(C')$
 - Let y be the first vertex discovered in C' . At the time $y.d$, all the vertices in C and C' are WHITE.
 - Any vertex in C' is a descendant of y .
 - From Lemma 22.13, $\nexists y \rightsquigarrow x, \forall x \in C$.
 - At the time $y.f$, C is still WHITE, so $f(C) > y.f = f(C')$.



Kosaraju's algorithm - Proof (4)

Corollary 22.15

Let C and C' be distinct SCC in directed graph $G = (V, E)$. Suppose there is an edge $(u, v) \in E^T$ where $u \in C$ and $v \in C'$. Then $f(C) < f(C')$.



Kosaraju's algorithm - Proof (4)

Corollary 22.15

Let C and C' be distinct SCC in directed graph $G = (V, E)$. Suppose there is an edge $(u, v) \in E^T$ where $u \in C$ and $v \in C'$. Then $f(C) < f(C')$.

- Since $(u, v) \in E^T$, we have $(v, u) \in E$.
- Since G and G^T have the same SCC, from *Lemma 22.14*
 $\Rightarrow f(C) < f(C')$.



Kosaraju's algorithm - Proof (5)

Theorem

The STRONGLY-CONNECTED-COMPONENTS procedure correctly computes the SCC of the directed graph G provided as input.



Kosaraju's algorithm - Proof (5)

Theorem

The STRONGLY-CONNECTED-COMPONENTS procedure correctly computes the SCC of the directed graph G provided as input.

- proof by induction on the number of DFS trees produced by calling DFS on G^T
- The induction hypothesis is that the trees $T_1, T_2, \dots, T_k$ are SCC and we need to prove that T_{k+1} is also a SCC. $k = 0$ is trivial.
- Let the root of T_{k+1} be u , that belongs to the SCC C .
- Since the DFS on G^T considers the vertices in descending order by finishing time, $u.f = f(C) > f(C')$ for any SCC C' that has yet to be visited.
- By the inductive hypothesis, at the time $u.d$, all nodes $v \in C$ are WHITE so they will be descendants of u in T_{k+1} . $\Rightarrow C \subseteq T_{k+1}$.
- From Corollary 22.15, any SCC C' that is reachable from C (in G^T) has $f(C) < f(C')$ so it has already been visited. $\Rightarrow T_{k+1} \subseteq C$.
- $C \subseteq T_{k+1} \wedge T_{k+1} \subseteq C \rightarrow T_{k+1} = C$, so T_{k+1} is an SCC.



Tarjan's algorithm

- the approach is also based on DFS
- keep a stack S (different from the call stack)
 - when the recursive visit returns, a node is not necessarily popped
 - a node u remains on the stack after being visited iff $\exists v \in S$, such that $\exists u \rightsquigarrow v$
- node attributes
 - $u.index$ – how many nodes were visited before the current one
 - $u.lowLink$ – smallest index of any node on the stack reachable from u through u 's DFS subtree
- if $u.lowLink == u.index$ (after the recursive visit) then u is the root of a new SCC

TARJAN(G)

```

1  global  $index = 0$ 
2  global  $S = \text{EMPTY-STACK}$ 
3  global  $nrComponents = 0$ 
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

```

STRONG-CONNECT(G, u)

```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11 if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

```

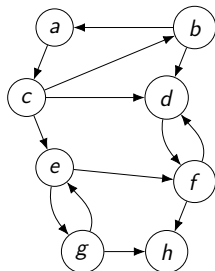
STRONG-CONNECT(G, u)

```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12     nrComponents = nrComponents + 1
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```

call stack



TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = FALSE$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

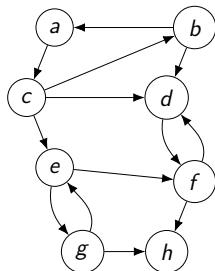
```

STRONG-CONNECT(G, u)

```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = TRUE$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12       $nrComponents = nrComponents + 1$ 
13      repeat
14           $v = POP(S)$ 
15           $v.onStack = FALSE$ 
16           $v.comp = nrComponents$ 
17      until  $v == u$ 

```

call stack S $index : 0$ $nrComponents : 0$ 

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = FALSE$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

```

STRONG-CONNECT(G, u)

```

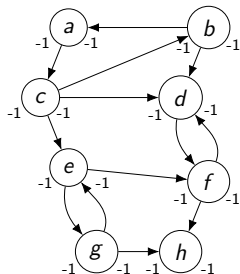
1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = TRUE$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = POP(S)$ 
15          $v.onStack = FALSE$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```

call stack S

index : 0

nrComponents : 0



TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

```

STRONG-CONNECT(G, u)

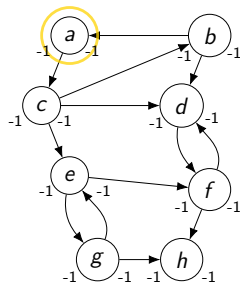
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack S



index : 0

nrComponents : 0

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

```

STRONG-CONNECT(G, u)

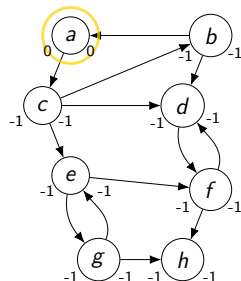
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack S



index : 0

nrComponents : 0



TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

```

STRONG-CONNECT(G, u)

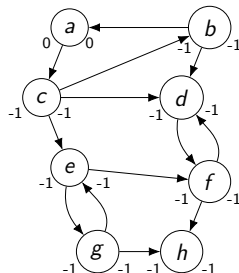
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12     nrComponents = nrComponents + 1
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack S



$index : 1$

$nrComponents : 0$

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

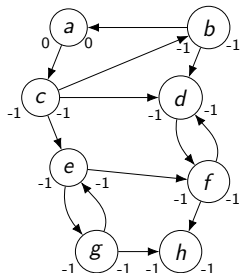
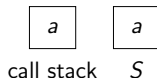
```

STRONG-CONNECT(G, u)

```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12     nrComponents = nrComponents + 1
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



index : 1

nrComponents : 0

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

```

STRONG-CONNECT(G, u)

```

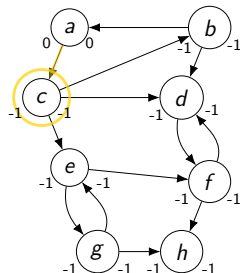
1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11 if  $u.lowLink == u.index$ 
12     nrComponents = nrComponents + 1
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack

S



index : 1

nrComponents : 0

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

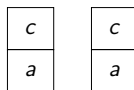
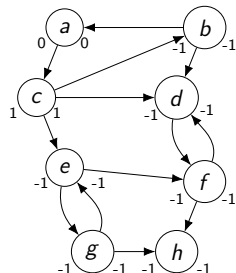
```

STRONG-CONNECT(G, u)

```

1   $u.index = u.lowLink = \text{index}$ 
2   $\text{index} = \text{index} + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12     nrComponents = nrComponents + 1
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```

call stack S  $\text{index} : 2$ $\text{nrComponents} : 0$

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

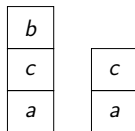
```

STRONG-CONNECT(G, u)

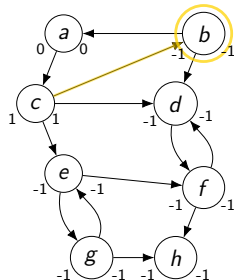
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack

 S  $index : 2$ $nrComponents : 0$

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

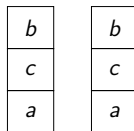
```

STRONG-CONNECT(G, u)

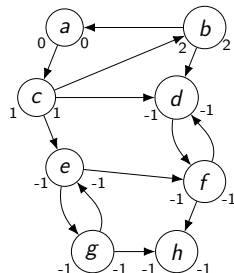
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack

 S  $index : 3$ $nrComponents : 0$

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

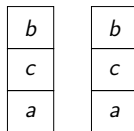
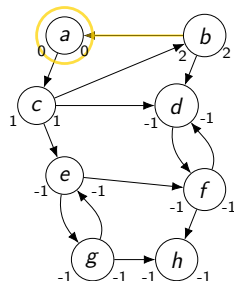
```

STRONG-CONNECT(G, u)

```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12     nrComponents = nrComponents + 1
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```

call stack S  $index : 3$ $nrComponents : 0$

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

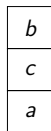
```

STRONG-CONNECT(G, u)

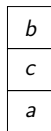
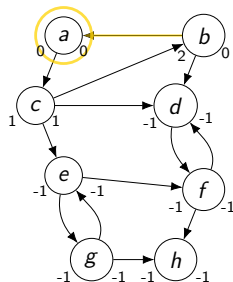
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12       $nrComponents = nrComponents + 1$ 
13      repeat
14           $v = \text{POP}(S)$ 
15           $v.onStack = \text{FALSE}$ 
16           $v.comp = nrComponents$ 
17      until  $v == u$ 

```



call stack

 S  $index : 3$ $nrComponents : 0$

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

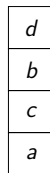
```

STRONG-CONNECT(G, u)

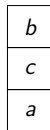
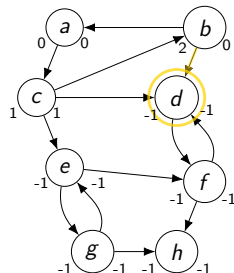
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack

 S  $index : 3$ $nrComponents : 0$

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

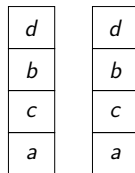
```

STRONG-CONNECT(G, u)

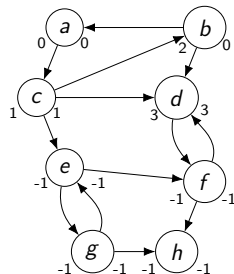
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12     nrComponents = nrComponents + 1
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack

 S  $index : 4$ $nrComponents : 0$

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

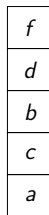
```

STRONG-CONNECT(G, u)

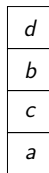
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12     nrComponents = nrComponents + 1
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

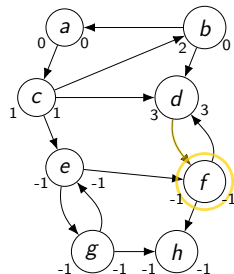
```



call stack



S



index : 4

nrComponents : 0

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

```

STRONG-CONNECT(G, u)

```

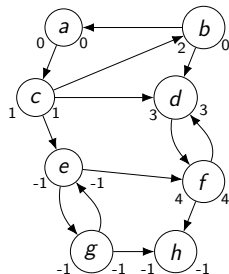
1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12     nrComponents = nrComponents + 1
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```

f
d
b
c
a

call stack

f
d
b
c
a

 S 

index : 5

nrComponents : 0

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = FALSE$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

```

STRONG-CONNECT(G, u)

```

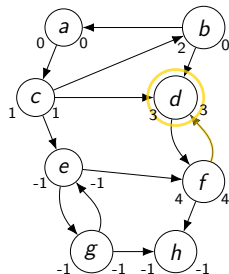
1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = TRUE$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12     nrComponents = nrComponents + 1
13     repeat
14          $v = POP(S)$ 
15          $v.onStack = FALSE$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```

f
d
b
c
a

call stack

f
d
b
c
a

 S 

index : 5

nrComponents : 0

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

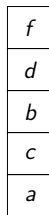
```

STRONG-CONNECT(G, u)

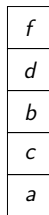
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      nrComponents = nrComponents + 1
13      repeat
14           $v = \text{POP}(S)$ 
15           $v.onStack = \text{FALSE}$ 
16           $v.comp = nrComponents$ 
17      until  $v == u$ 

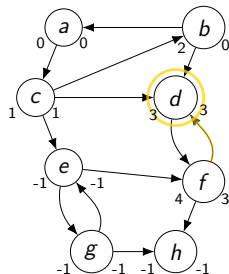
```



call stack



S



index : 5

nrComponents : 0

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

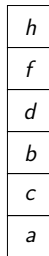
```

STRONG-CONNECT(G, u)

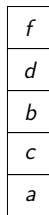
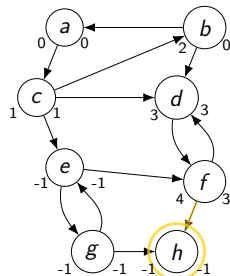
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11 if  $u.lowLink == u.index$ 
12     nrComponents = nrComponents + 1
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack

 S 

index : 5

nrComponents : 0

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

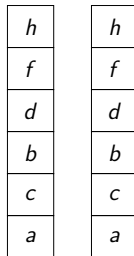
```

STRONG-CONNECT(G, u)

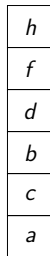
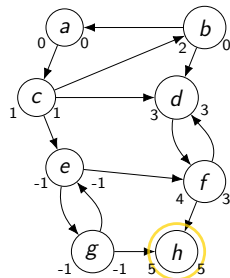
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12     nrComponents = nrComponents + 1
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack

 S  $index : 6$ $nrComponents : 0$

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = FALSE$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

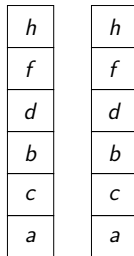
```

STRONG-CONNECT(G, u)

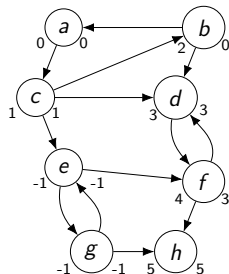
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = TRUE$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12     nrComponents = nrComponents + 1
13  repeat
14      $v = POP(S)$ 
15      $v.onStack = FALSE$ 
16      $v.comp = nrComponents$ 
17  until  $v == u$ 

```



call stack

 S  $index : 6$ $nrComponents : 1$

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

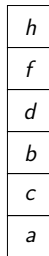
```

STRONG-CONNECT(G, u)

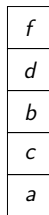
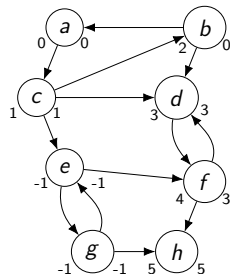
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack

 S  $index : 6$ $nrComponents : 1$

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

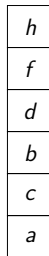
```

STRONG-CONNECT(G, u)

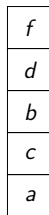
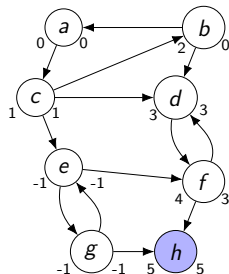
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack

 S  $index : 6$ $nrComponents : 1$

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = FALSE$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

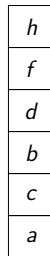
```

STRONG-CONNECT(G, u)

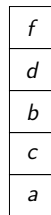
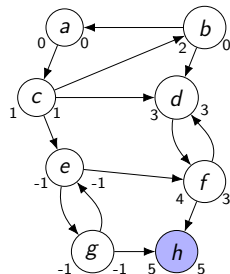
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = TRUE$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = POP(S)$ 
15          $v.onStack = FALSE$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack

 S  $index : 6$ $nrComponents : 1$

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = FALSE$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

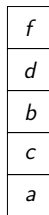
```

STRONG-CONNECT(G, u)

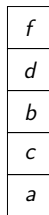
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = TRUE$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12     nrComponents = nrComponents + 1
13     repeat
14          $v = POP(S)$ 
15          $v.onStack = FALSE$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

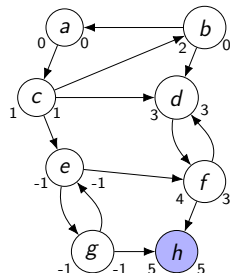
```



call stack



S



index : 6

nrComponents : 1

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

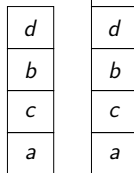
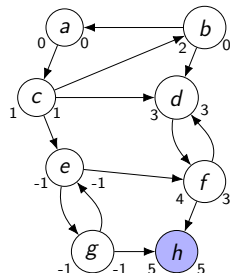
```

STRONG-CONNECT(G, u)

```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12     nrComponents = nrComponents + 1
13  repeat
14      $v = \text{POP}(S)$ 
15      $v.onStack = \text{FALSE}$ 
16      $v.comp = nrComponents$ 
17  until  $v == u$ 

```

call stack S  $index : 6$ $nrComponents : 2$

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

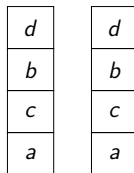
```

STRONG-CONNECT(G, u)

```

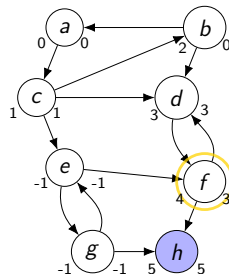
1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack

S



index : 6

nrComponents : 2

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

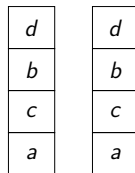
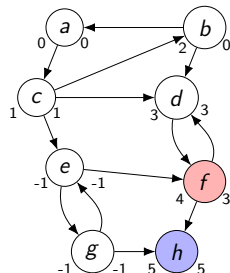
```

STRONG-CONNECT(G, u)

```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```

call stack S  $index : 6$ $nrComponents : 2$

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = FALSE$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

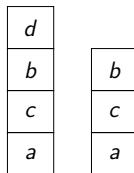
```

STRONG-CONNECT(G, u)

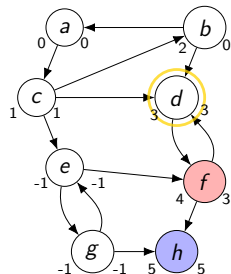
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = TRUE$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = POP(S)$ 
15          $v.onStack = FALSE$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack

 S  $index : 6$ $nrComponents : 2$

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

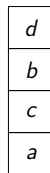
```

STRONG-CONNECT(G, u)

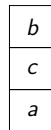
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

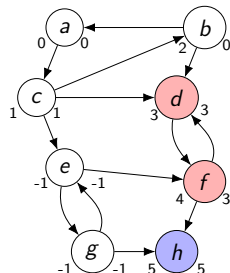
```



call stack



S



index : 6

nrComponents : 2

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

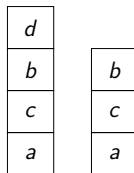
```

STRONG-CONNECT(G, u)

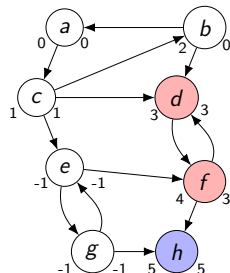
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack

 S  $index : 6$ $nrComponents : 2$

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

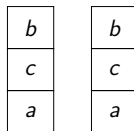
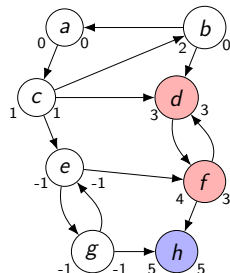
```

STRONG-CONNECT(G, u)

```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12     nrComponents = nrComponents + 1
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```

call stack S  $index : 6$ $nrComponents : 2$

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

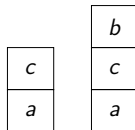
```

STRONG-CONNECT(G, u)

```

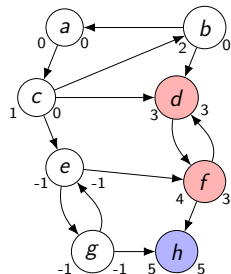
1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack

S



index : 6

nrComponents : 2

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

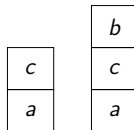
```

STRONG-CONNECT(G, u)

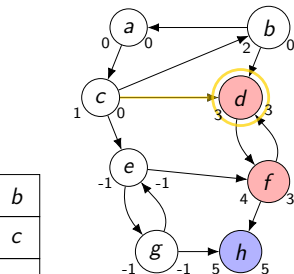
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12     nrComponents = nrComponents + 1
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack



index : 6

nrComponents : 2

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

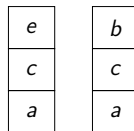
```

STRONG-CONNECT(G, u)

```

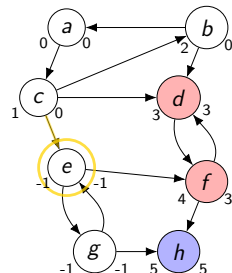
1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack

S



index : 6

nrComponents : 2

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = FALSE$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

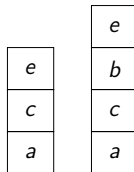
```

STRONG-CONNECT(G, u)

```

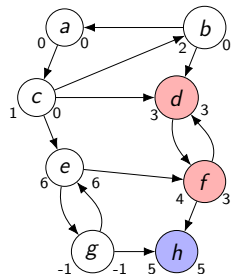
1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = TRUE$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = POP(S)$ 
15          $v.onStack = FALSE$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack

S



index : 7

nrComponents : 2

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

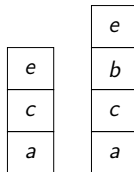
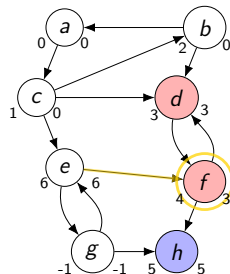
```

STRONG-CONNECT(G, u)

```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```

call stack S  $index : 7$ $nrComponents : 2$

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

```

STRONG-CONNECT(G, u)

```

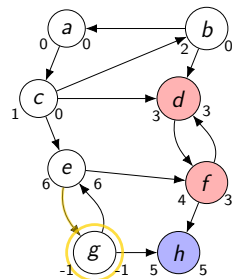
1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```

g
e
c
a

call stack

e
b
c
a

 S  $index : 7$ $nrComponents : 2$

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

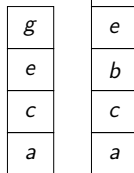
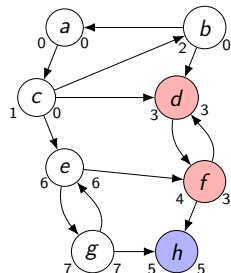
```

STRONG-CONNECT(G, u)

```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12     nrComponents = nrComponents + 1
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```

call stack S  $index : 8$ $nrComponents : 2$

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

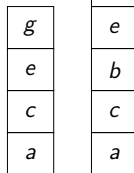
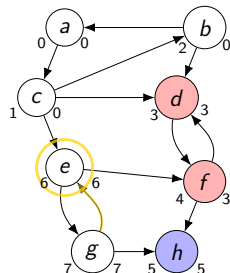
```

STRONG-CONNECT(G, u)

```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12     nrComponents = nrComponents + 1
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```

call stack S  $index : 8$ $nrComponents : 2$

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = FALSE$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

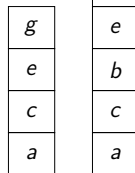
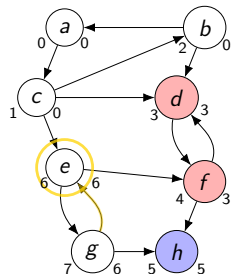
```

STRONG-CONNECT(G, u)

```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = TRUE$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12       $nrComponents = nrComponents + 1$ 
13      repeat
14           $v = POP(S)$ 
15           $v.onStack = FALSE$ 
16           $v.comp = nrComponents$ 
17      until  $v == u$ 

```

call stack S  $index : 8$ $nrComponents : 2$

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

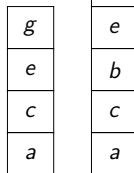
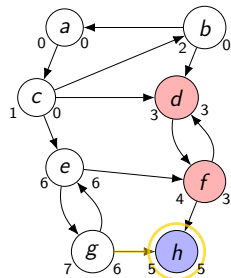
```

STRONG-CONNECT(G, u)

```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```

call stack S  $index : 8$ $nrComponents : 2$

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

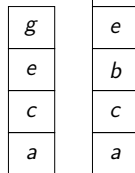
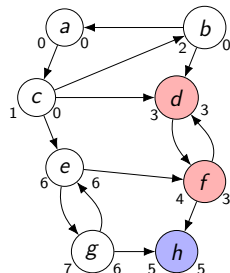
```

STRONG-CONNECT(G, u)

```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12     nrComponents = nrComponents + 1
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```

call stack S  $index : 8$ $nrComponents : 2$

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

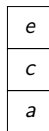
```

STRONG-CONNECT(G, u)

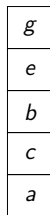
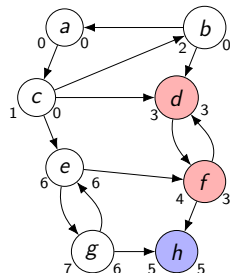
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12     nrComponents = nrComponents + 1
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack

 S 

index : 8

nrComponents : 3

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

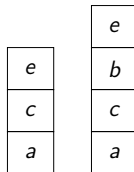
```

STRONG-CONNECT(G, u)

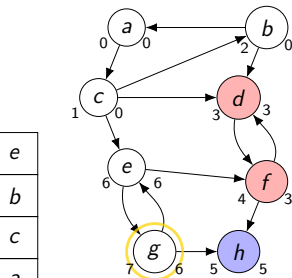
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack



index : 8

nrComponents : 3

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

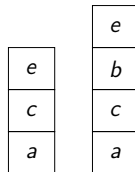
```

STRONG-CONNECT(G, u)

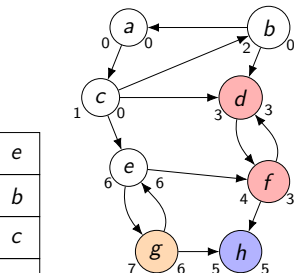
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack



index : 8

nrComponents : 3



TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

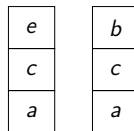
```

STRONG-CONNECT(G, u)

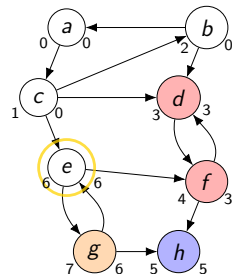
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12     nrComponents = nrComponents + 1
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack S



index : 8

nrComponents : 3

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

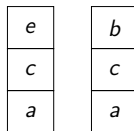
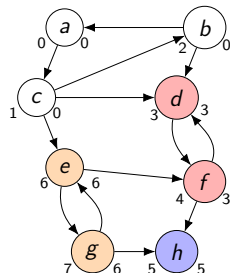
```

STRONG-CONNECT(G, u)

```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```

call stack S  $index : 8$ $nrComponents : 3$

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

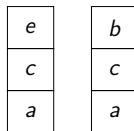
```

STRONG-CONNECT(G, u)

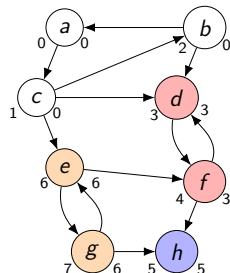
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack

 S 

index : 8

nrComponents : 3

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

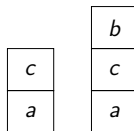
```

STRONG-CONNECT(G, u)

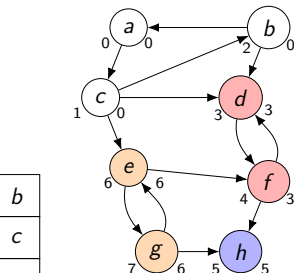
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12     nrComponents = nrComponents + 1
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack



index : 8

nrComponents : 3

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

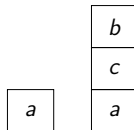
```

STRONG-CONNECT(G, u)

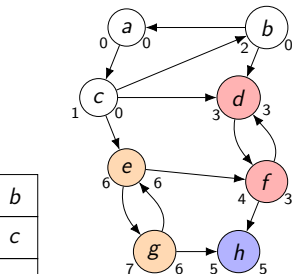
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12     nrComponents = nrComponents + 1
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack



index : 8

nrComponents : 4

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

```

STRONG-CONNECT(G, u)

```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

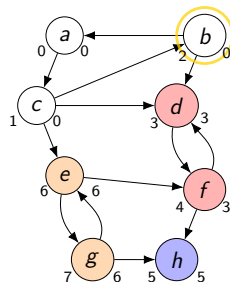
```



call stack



S



index : 8

nrComponents : 4

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

```

STRONG-CONNECT(G, u)

```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

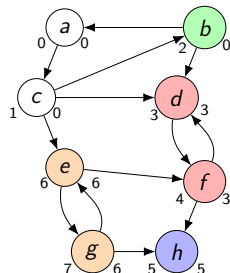
```



call stack



S



index : 8

nrComponents : 4

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = FALSE$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

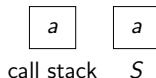
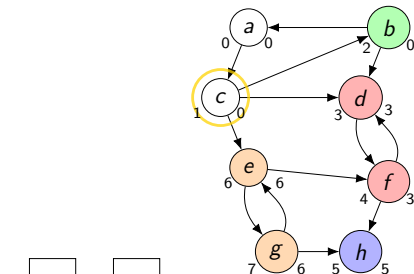
```

STRONG-CONNECT(G, u)

```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = TRUE$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = POP(S)$ 
15          $v.onStack = FALSE$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



index : 8

nrComponents : 4

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

```

STRONG-CONNECT(G, u)

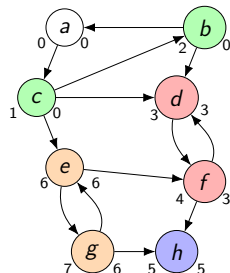
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack S



index : 8

nrComponents : 4

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

```

STRONG-CONNECT(G, u)

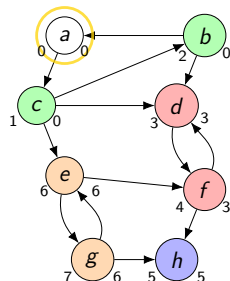
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack S



index : 8

nrComponents : 4

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

```

STRONG-CONNECT(G, u)

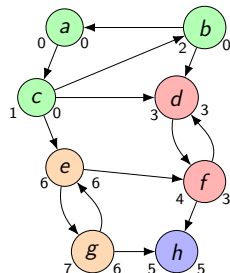
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack S



index : 8

nrComponents : 4

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

```

STRONG-CONNECT(G, u)

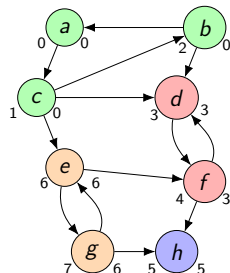
```

1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11  if  $u.lowLink == u.index$ 
12      $nrComponents = nrComponents + 1$ 
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```



call stack S



index : 8

nrComponents : 4

TARJAN(G)

```

1  global index = 0
2  global S = EMPTY-STACK
3  global nrComponents = 0
4  for each vertex  $u \in G.V$ 
5       $u.index = u.lowLink = -1$ 
6       $u.onStack = \text{FALSE}$ 
7       $u.comp = 0$ 
8  for each vertex  $u \in G.V$ 
9      if  $u.index == -1$ 
10         STRONG-CONNECT( $G, u$ )

```

STRONG-CONNECT(G, u)

```

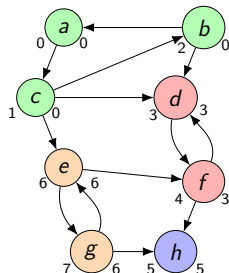
1   $u.index = u.lowLink = index$ 
2   $index = index + 1$ 
3  PUSH( $S, u$ )
4   $u.onStack = \text{TRUE}$ 
5  for each  $v \in G.Adj[u]$ 
6      if  $v.index == -1$ 
7          STRONG-CONNECT( $G, v$ )
8           $u.lowLink = \min(u.lowLink, v.lowLink)$ 
9      elseif  $v.onStack$ 
10          $u.lowLink = \min(u.lowLink, v.index)$ 
11 if  $u.lowLink == u.index$ 
12     nrComponents = nrComponents + 1
13     repeat
14          $v = \text{POP}(S)$ 
15          $v.onStack = \text{FALSE}$ 
16          $v.comp = nrComponents$ 
17     until  $v == u$ 

```

call stack S

index : 8

nrComponents : 4





Bibliography

- Cormen, Thomas H., et al., *"Introduction to algorithms."*, MIT press, 2009, cap. 22.3, 22.4, 22.5